

On the use of usage patterns from telemetry data for test case prioritization

Jeff Anderson^{a,*}, Maral Azizi^{b,*}, Saeed Salem^c, Hyunsook Do^b

^a Microsoft Inc., Fargo, North Dakota, United States

^b University of North Texas, Denton, TX, United States

^c North Dakota State University, Fargo, ND, United States

ARTICLE INFO

Keywords:

Regression testing
Test case prioritization
Telemetry data
Usage patterns

ABSTRACT

Context: Modern applications contain pervasive telemetry to ensure reliability and enable monitoring and diagnosis. This presents a new opportunity in the area of regression testing techniques, as we now have the ability to consider usage profiles of the software when making decisions on test execution.

Objective: The results of our prior work on test prioritization using telemetry data showed improvement rate on test suite reduction, and test execution time. The objective of this paper is to further investigate this approach and apply prioritization based on multiple prioritization algorithms in an enterprise level cloud application as well as open source projects. We aim to provide an effective prioritization scheme that practitioners can implement with minimum effort. The other objective is to compare the results and the benefits of this technique factors with code coverage-based prioritization approaches, which is the most commonly used test prioritization technique.

Method: We introduce a method for identifying usage patterns based on telemetry, which we refer to as “telemetry fingerprinting.” Through the use of various algorithms to compute fingerprints, we conduct empirical studies on multiple software products to show that telemetry fingerprinting can be used to more effectively prioritize regression tests.

Results: Our experimental results show that the proposed techniques were able to reduce over 30% in regression test suite run times compared to the coverage-based prioritization technique in detecting discoverable faults. Further, the results indicate that fingerprints are effective in identifying usage patterns, and that the fingerprints can be applied to improve regression testing techniques.

Conclusion: In this research, we introduce the concept of fingerprinting software usage patterns through telemetry. We provide various algorithms to compute fingerprints and conduct empirical studies that show that fingerprints are effective in identifying distinct usage patterns. By applying these techniques, we believe that regression testing techniques can be improved beyond the current state-of-the-art, yielding additional cost and quality benefits.

1. Introduction

Regression tests are among the most important tools for ensuring code quality in subsequent product releases [9]. These tests are run prior to releasing a changed version of a product in order to catch any defects before users are impacted. As regression test suites grow larger over time, regression testing techniques such as regression test selection, test suite reduction, and test case prioritization [23,50] are commonly used to manage the increased cost of executing large suites of tests. Typically, these techniques focus on attributes such as code churn¹ to identify areas likely to have defects, so testing can focus on those areas first. What has *not* been considered in typical regression testing techniques is the

likelihood of a user encountering a given defect [20,43]. This means that the traditional application of regression testing techniques treats all faults as equally likely to be discovered, focusing only on whether those faults are detected by regression testing, or by how serious the damage would be if the fault were encountered by a user.

In practice, however, large, complex software systems often contain a large number of potential faults, many of which are never encountered by any users. For instance, Hatton [29] reported around 6 faults per KLOC in “good” software, with a belief that under optimum conditions this could be decreased closer to 1 fault per KLOC. This is further confirmed by Controneo et al. [14], showing between 0.35 and 5 faults per KLOC in major open-source products. Applying these rates to complex software systems with 10 million lines of code or more, that indicates a likely best case number of faults around 3500 to 60,000. Granted, many of these faults are exceedingly minor annoyances, but

* Corresponding authors.

E-mail addresses: jeffand@microsoft.com (J. Anderson), marazazizi@my.unt.edu, maralazizi@my.unt.edu (M. Azizi), saeed.salem@ndsu.edu (S. Salem), hyunsook.do@unt.edu (H. Do).

¹ Code churn is a metric that can measure change volume between two versions of a software, e.g. lines of code added, lines of code deleted, etc.

they are still faults. This research seeks to better focus detection and fixing of those smaller number of faults which are noticeable and impacting customers. If a fault exists but never impacts a user, then we hypothesize that this fault has less impact on the overall system reliability, therefore, those faults that are impacting users has a higher impact on the users perspective of the system functionality and need to be fixed prior to other faults. In this research, we refer these faults as discoverable faults, which exist in software and impact at least one user of the software. Faults by themselves are not able to be categorized as discoverable or non-discoverable until after users make use of the software. Any faults exercised during the usage of the software become discoverable, while the remaining faults that were not executed by any users remain non-discoverable.

An important trend in software that enables this distinction to be drawn is the pervasive use of telemetry in software products. As software is moving to Software-as-a-Service (SaaS), software vendors are forced to add telemetry throughout the product in order to monitor and diagnose issues that arise at runtime, since live services cannot be directly debugged². Telemetry is data collected at a distance from actual product usage. This differs from traditional software metrics such as complexity or churn that are available at software design time independent of how the software is used.

The pervasive telemetry for monitoring and diagnosis provides a new avenue for categorizing faults as discoverable or non-discoverable. After a fault is found (by a user, internal manual analysis, fault detection tooling, or other techniques), then usage telemetry can be consulted to determine whether or not any user has ever been impacted by that fault. The fault is only discoverable if at least one user has ever been impacted. We feel that a new opportunity has emerged in regression testing based on the ability to classify faults as discoverable or not through telemetry. Similar to how genome sequencing has given rise to the new field of personalized medicine [27], we now have the ability to create regression testing processes personalized to the usage patterns of a specific service or customer. By focusing on the actual usage of the software and prioritizing discoverable faults, economic and reliability benefits can be gained through reduced testing time and the discovery of faults that are more likely to impact users.

To address discoverable faults more effectively, we introduce a new concept we refer to as “telemetry fingerprinting”. A telemetry fingerprint is the unique usage pattern of a specific software service or by a specific customer of a service. We believe that software usage tends to follow repeated, predictable patterns for different groups of users. We also believe that these patterns can be used to predict which faults will be more likely to be discoverable by that group of users. Telemetry fingerprinting can then be combined with traditional regression testing techniques such as test suite prioritization. In this case, tests that are more likely to uncover discoverable faults (as opposed to any faults) can be prioritized first. Since discoverable faults will have more impact on users of the software, it is more important to detect these faults than non-discoverable faults.

In this work, our goal is to examine the applicability of usage fingerprinting in the area of test case prioritization. To investigate, we performed two empirical studies. The first study is based on a prerelease industrial product, *Microsoft Dynamics AX*. The second experiment repli-

cates our first, using two web applications, *CarrotCakeCMS* and *DASCP*. These two projects are added to further evaluate the proposed technique under an open source project domain utilized by non professional users. Through these studies, we show that applying telemetry fingerprinting to regression suite prioritization can reduce the time taken to find all discoverable faults by over 30% compared to a coverage-based technique.

This paper is substantially expanded from our previously published paper [5]. The new aspects are highlighted as follows:

1. *A replicated study*: We replicated the study conducted from the previous publication using new web applications to ensure that the findings generalize across products.
2. *Evaluating similarity function effectiveness*: We have added additional research question to determine which similarity functions are most effective in fingerprinting usage, both in the original dataset as well as in the replicated study.
3. *Additional data and analysis*: This paper provides additional data and analysis that demonstrate the underlying usage patterns of software and help illustrate why these techniques are effective.

The rest of the paper is organized as follows. In [Section 2](#), we discuss the approach used in the research as well as formally define telemetry fingerprinting. In [Sections 3](#) and [4](#), we detail the empirical studies that we performed including the results of the studies. [Section 5](#) details threats to validity. [Section 6](#) discusses the implications of our results, and [Section 7](#) presents related work. Finally, in [Section 8](#), we provide conclusions and discuss future work.

2. Proposed approach

2.1. Motivation

Products with wide functionality footprints such as *Microsoft Dynamics AX* are often characterized by very different levels of usage for different areas of functionality. Rate of usage also differs significantly according to the type of customer using the product. For instance, a large enterprise resource planning (ERP) project may have modules for managing warehouses, tracking sales orders and managing financial results. A customer using the product who is in the professional services industry may use the sales order tracking function heavily while not using the warehouse functionality at all. Similarly, a logistics company may rely heavily on warehousing functionality while tracking financial results in an entirely different system.

Large-scale ERP and other products usually have extremely large regression test suites to cover the broad range of functionality they support. The cost of executing these suites can also be significant in terms of both time and computer resources. To reduce costs and gain economic benefits, regression testing techniques such as test case prioritization are often applied. These techniques traditionally do not consider usage information when prioritizing test cases. Rather, regression testing techniques rely on aspects such as churn to predict which tests are most likely to uncover faults.

In this research, we seek to introduce usage information as an additional input to traditional regression testing techniques in order to improve the results. Usage information has traditionally been expensive and difficult to obtain. However, as many products are shifting to Software-as-a-Service (SaaS) delivery, telemetry is becoming pervasive. This telemetry is necessary in order to monitor and maintain SaaS. Given the availability of telemetry that provides usage information, the inclusion of such information in regression testing techniques is a natural evolution in this area of research.

2.2. Discoverable faults

Not all faults are equal. In addition to traditional fault severity classification, faults can also be classified by whether or not any users ever

² Telemetry can be collected from any software product. However, we see practically in the field that more traditional “on premise” software typically contains little if any telemetry. With newer versions of Mac OS, Windows, and others, the amount of telemetry collected is increasing, and is a valid application of this research. The reason we call out Software-as-a-Service specifically, is in SaaS environments the traditional debugging and troubleshooting techniques like tracing are no longer possible because the software instances no longer run in isolation. As such, providers of SaaS applications have been at the forefront of pushing robust telemetry into their application and collecting it in usable stores because that is the *only* data source available when issues arise. It is this trend, driven most completely by SaaS applications, which gave rise to our research.

encounter them. As mentioned in the Section 2.1, there is wide variability in the usage of software products. Some customers may never even use an entire area of a product. If a fault exists in that unused part of the product, then it has no negative impact on that customer. On the other hand, if a fault exists in an area of the product that a customer is using heavily, then the impact of that fault on that customer is much greater.

To address this classification, we introduce the concept of “discoverable faults.” Discoverable faults are those that will impact one or more users of the software. If a fault exists but no user is ever impacted by it, that fault is less important than faults which are actively impacting users.

A fault may exist and not impact users much without being dead code. An example may be a simple performance fault related to opening a setup form that is used once during configuration of the system and then never again. A one-second delay on form open in the setup form may not even be noticed by a user and have no impact. Conversely, that same one-second delay while opening the sales order or posting form would have a major impact on the perceived performance of the application and a wide range of users. In this trivial example, we know the setup form is exceedingly rarely used and can proactively de-prioritize work on really minor fault in this area. In other areas of the product, our understanding of which feature areas are more heavily used is less clear. In those areas where we do not have a clear understanding of usage, we use telemetry data to inform where we should apply additional investments. Investments based on usage includes fixing faults, and also investments like more automate test coverage and manual fault discovery.

Research on regression testing techniques traditionally uses all faults as a ground truth for determining effectiveness [18,44,46]. The goal of regression testing techniques should be to reduce the likelihood that a user will encounter a fault, not just to reduce the total faults in the system. In this research, we are able to use telemetry data to determine if any users have exercised a given fault after the release of the product. It is the total number of “discoverable faults” that we wish to reduce.

2.3. Approach

2.3.1. Computing fingerprint scores

In this section, we describe the approach used in fingerprinting data processes and tests, and how that fingerprint is utilized in regression testing techniques. This research was done using a prerelease version of *Microsoft Dynamics AX*. This version was installed and used in a live environment by multiple companies. Telemetry was collected from these installations.

In this research, we use the term *telemetry fingerprint* or just *fingerprint* to mean the unique product usage signature derived from telemetry information. In all telemetry sessions, the product and version are both the same, containing the same code. The difference is in the usage of that product in a given installation. For instance, a distribution company will have a very different pattern of usage from a financial services company, focusing on very different areas of the product. We call these differences in usage *fingerprints*.

Fig. 1 shows an example of the raw telemetry data. The first column is the session identifier, which identifies a single user navigating through the product. The second column is the set of distinct interactions performed by the user that caused a call to the server. The syntax of the interactions is of the format *(Form name):(Control name):(Command name)*.

Using the example from Fig. 1, the telemetry indicates that the user performed the following steps or interactions:

- The user started on the default dashboard.
- The user clicked on the fixed assets form and launched it.
- While the fixed assets form was loading, a number of form parts loaded.
- The user clicked in the main grid on the fixed assets form.

We know this because the first telemetry action was `DefaultDashboard:DefaultDashboard:Init`. This means the default dashboard form being initialized was the first action in this telemetry stream. Following that we see `AssetTable:AssetTable:init`, indicating initialization of the `AssetTable` form was the second action the user performed. In this same way, we understand the rest of the steps being performed by the user.

A collection of sessions similar to the one shown in Fig. 1 exists in telemetry for each installation. In this article, we are trying to determine whether the collection of sessions provides a unique fingerprint for that installation. To determine if a fingerprint exists, we need a way to determine how similar two sessions are. A variety of different session similarity functions are discussed later in this article (Section 2.4).

Let $I = \{i_1, i_2, \dots, i_m\}$ denote the finite set of events (user interactions). An “event” is a generic term referring to a user action that the tool is capturing, whether launching a form, clicking a button, or changing a field. A user session or navigation path is represented as a sequence that is an ordered list of events. A single session comes from a single user and a single installation of the software product. A single session, S , is modeled as a sequence that is an ordered list of user interactions:

$$S = \{a_1, a_2, \dots, a_l\}$$

where $a_i \in I$ is a user interaction and l is the length of the session.

We define a set of sessions G_i to be an unordered collection of sessions from the same installation of the software product:

$$G_i = \{S_1, S_2, \dots, S_m\}$$

where S_i is a session that belongs to the installation, and m is the number of sessions in the installation.

The union of all the sessions from all the installations is denoted as D and is defined as follows:

$$D = \bigcup_{i=1}^c G_i$$

where c is the total number of installations.

To determine fingerprints, we need a measure to quantify the similarity between sessions. We introduce a similarity function that assigns a real value to quantify the similarity between two sessions, $O : D \times D \rightarrow \mathbb{R}$. These similarity functions are described in detail in the following

fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	DefaultDashboard:DefaultDashboard:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetTable:AssetTable:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetNetBookValuePart:AssetNetBookValuePart:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetDepreciationPart:AssetDepreciationPart:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetAcquisitionInfoPart:AssetAcquisitionInfoPart:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetAdditionsInfoPart:AssetAdditionsInfoPart:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetProjectInfoPart:AssetProjectInfoPart:Init
fdbed8fc-4fea-45b4-ac9b-cde21cc8a7e8	AssetTable:Grid:SetClientViewStart

Fig. 1. Sample interaction session.

section. Given two sessions from the domain D , a similarity function returns a value between 0 and 1 describing how similar the sessions are to each other. A value of 0 denotes they are not at all similar, while a value of 1 denotes they have high similarity. The similarity functions may compute the similarity based on different attributes of the sessions, as described in the next section.

In the following equation, $O(S_j, S_k)$ denotes the similarity of S_j and S_k . For sessions in the same installation, we introduce two similarity measurements: internal and external. The internal similarity measures the average pairwise session similarity of all the sessions in the same installation, which captures the homogeneity of the sessions in an installation. For an installation with m_i sessions, we define the internal similarity as follows:

$$F_{int}(G_i) = \frac{2}{m_i \times (m_i - 1)} \sum_{S_j \in G_i} \sum_{S_k \in G_i, k > j} O(S_j, S_k)$$

where m_i is the number of sessions in installation i . Installations with similar user sessions will have high internal similarity.

The external similarity of an installation measures the average similarity between the sessions in an installation and all the sessions in the other installations.

$$F_{ext}(G_i) = \frac{1}{m_i \times |D \setminus G_i|} \sum_{S_j \in G_i} \sum_{S_k \in D \setminus G_i} O(S_j, S_k)$$

To measure how similar the sessions in an installation compared to the sessions in the other installations, we introduce a fingerprint score, FP , for an installation. The fingerprint score is the ratio of the internal similarity to the external similarity of an installation.

$$FP(G_i) = \frac{F_{int}(G_i)}{F_{ext}(G_i)}$$

Installations with high fingerprint scores have high similarity between sessions within the installation but low similarity to sessions in different installations. We hypothesize that similarity functions that yield a high FP value will be most effective at identifying usage patterns. In this research, we seek to find session similarity functions that produce high scores of FP for installations.

2.3.2. Evaluating fingerprint effectiveness

Given a single session, we employ the similarity score to identify the installation that the session belongs to. To test the effectiveness of the installation prediction, we randomly select n sessions from an installation and try to predict the installation of these sessions. Let R_i denote the randomly selected sessions from G_i and R'_i denote the set of the remaining sessions, i.e., $R_i \subseteq G_i$, and $R'_i = G_i \setminus R_i$.

For each session in $S_j \in R_i$, we compute the membership score, M_i , in installation i as the average similarity of the session to the sessions in

installation. The membership score is calculated as follows:

$$M_i(S_j) = \frac{1}{|R'_i|} \sum_{S_k \in R'_i} O(S_j, S_k)$$

We predict the installation for session, S_j , as the installation that maximizes the membership score M_i .

$$Installation(S_j) = \arg \max_k M_k(S_j)$$

The rationale for this prediction method is that sessions that belong to the same installation have higher similarity. Since we know the true installation of the session, we can then compute the accuracy (effectiveness) of the fingerprinting score.

2.4. Session similarity functions

In this section, we describe various similarity functions in detail using examples. In these examples, we will use the sample sessions shown in Fig. 2.

In this example, Session 1 shows a user starting from the main dashboard, launching the sales order form, switching rows, filling in a field, then switching rows again. Similarly, Session 2 shows a user starting from the dashboard, going to the fixed assets form, opening the depreciation dialog, and clicking “OK”. Session 3 starts from the dashboard, opens the purchase order form, and confirms a purchase order. Session 4 again starts from the dashboard and opens the sales order form, but this time closes the enhanced preview and then switches rows twice.

2.4.1. By length

The similarity between two sessions is based on the lengths of these sessions by taking the length of the shorter session and dividing by the length of the longer session, i.e., $O(S_i, S_j) = \frac{\min(|S_i|, |S_j|)}{\max(|S_i|, |S_j|)}$. So, for this example, Session 3 has three events and Session 1 has five, giving a ratio of $3/5 = 0.6$. Using the same approach, we get $O(S_2, S_3) = 3/4 = 0.75$, $O(S_1, S_4) = 5/5 = 1$, and $O(S_2, S_4) = 4/5 = 0.8$.

2.4.2. By Form Match

Form Match is the ratio of forms from the shorter session that are present in the longer session. Using Fig. 2 as an example, the length of Session 3 is shorter than the length of Session 1. The first form in Session 3 is the *Dashboard*, which exists in Session 1. The second is *PurchTable*, which does not exist in Session 1. The third is again *PurchTable*, which does not exist in Session 1. The overall score for $O(S_3, S_1) = (1 + 0 + 0)/3 = 0.333$.

Using the same technique for calculating the similarities, $O(S_1, S_4)$, $O(S_2, S_3)$, and $O(S_2, S_4)$, we get scores of 1, 0.333, and 0.25, respectively.

Base Sessions

1: Dashboard:Init
SalesTable:Init
SalesTable:Grid:SelectRow
SalesTable:ItemId:SetValue
SalesTable:Grid:SelectRow

2: Dashboard:Init
AssetTable:Init
AssetTable:Depreciation:Click
DepreciationDialog:OkButton:Click

Test Sessions

3: Dashboard:Init
PurchTable:Init
PurchTable:Confirm:Click

4: Dashboard:Init
SalesTable:Init
SalesTable:EnhancedPreview:Close
SalesTable:Grid:SelectRow
SalesTable:Grid:SelectRow

Fig. 2. Examples of session data.

2.4.3. By Action Match

Action Match scoring is nearly identical to Form Match, only the full action is compared as opposed to just the form name. Two sessions can have high similarity in the forms, yet contain very different action sets. When comparing Sessions 1 and 4 by Form Match we compute a score of 1, but Action Match only scores 0.8 because the *SalesTable:ItemId:SetValue* action is not found in Session 4. Completing this example, we get the following scores: $O(S_1, S_3) = 0.33$, $O(S_1, S_4) = 0.8$, $O(S_2, S_3) = 0.333$, and $O(S_2, S_4) = 0.25$.

2.4.4. By E-Count

This technique is arguably very naïve, but is included to show the power of this fingerprinting approach. In this technique, the ratios of the number of occurrences of the letter *E* from one session to another session are compared. In Session 1, the letter *E* occurs fifteen times (twice in *SalesTable:Init*, four times in *SalesTable:Grid:SelectRow* and so on). It occurs eight times in Session 2, twice in Session 3, and seventeen times in Session 4.

The ratio of smaller to larger count in each comparison of 1/3, 1/4, 2/3, and 2/4 yields a fingerprint score of: $((2/14) + (14/17) + (2/8) + (8/17))/4 = 0.422$.

While this technique may appear somewhat random, note that the number of times the letter *E* occurs in a given session is somewhat related to which forms are in use in that session. *SalesTable* has two, while *PurchTable* only has one. This means that even a simplistic technique like counting letters will encode some information about whether or not the same forms are being used in two sessions.

2.4.5. By Markov Chain

In Markov Chain technique, we are looking for the probability of observing two sets of similar sessions with a correlation. The output is a matrix that shows how many times each navigation was observed in the base sessions. This matrix serves as the base emission matrix for the Markov Chain. Fig. 3 shows the initial emission matrix.

In this figure, the cells that contain the value 1 indicate that there is a transition between the two elements, and the cells that do not contain any value indicate that there is no transition between the two elements.

	Page-load: Init	PageImport: Init	PageImport: Init: btnUpload:Click	SiteExport:Init	SiteSkinEdit: Init:btnApply:Click	SiteSkinEdit: Init:btnDelete:Click	SiteExport:Init
Page-load: Init		1			1		
PageImport: Init			1				
PageImport: Init: btnUpload:Click				1			
SiteExport:Init			1				
SiteSkinEdit: Init:btnApply:Click						1	
SiteSkinEdit: Init:btnDelete:Click							1
SiteExport:Init							

Table 1

Example pairwise similarities.

Session 1	Session 2	Pair-wise similarity
S_a	S_b	0.786
S_a	S_c	0.627
S_a	S_d	0.423
S_a	S_e	0.400
S_a	S_f	0.128
S_b	S_c	0.658
S_b	S_d	0.351
S_b	S_e	0.109
S_b	S_f	0.220
S_c	S_d	0.423
S_c	S_e	0.726
S_c	S_f	0.101
S_d	S_e	0.682
S_d	S_f	0.841
S_e	S_f	0.773

As shown in Fig. 3, for some cells, there are no transitions in the base sessions but transitions exist in the test sessions. For example, the navigation from the *Page-load:Init* to *UserGroupAddEdit:btnAppy:Click* does not exist at all in the base sessions. Thus, we need to make a correction; otherwise, the probability results would be always zero for most cases. In order to address this issue, we added a small value to these cells. To avoid skewing the navigation ratios, that same value of 0.1 is also added to the two other navigations. The same process was applied to all missing navigation. Fig. 4 shows the corrected matrix.

2.5. Example

This section provides an example of the computations explained in the previous section. Suppose we have two datasets, G_1 and G_2 .

For each of these datasets, we have three sessions of data, labeled $G_1 = \{S_a, S_b, S_c\}$ and $G_2 = \{S_d, S_e, S_f\}$. This is a total of six sessions. Further suppose that the algorithm being used for fingerprinting yields the pairwise similarity measurements listed in Table 1.

Fig. 3. Initial emission matrix.

	Page-load: Init	PageImport: Init	PageImport: btnUpload:Click	SiteExport:Init	SiteSkinEdit: btnApply:Click	SiteSkinEdit: btnDelete:Click	SiteExport:Init	UserMembership: Init	UserGroupAddEdit:Init:btnApply:Click	TagAddEdit
Page-load: Init		1.1			1.1			0.1		
PageImport: Init			1.1							0.1
PageImport: btnUpload:Click			0.1	1.1						
SiteExport:Init			1							
SiteSkinEdit: btnApply:Click						1				
SiteSkinEdit: btnDelete:Click							1			
SiteExport:Init										
UserMembership: Init									0.1	
UserGroupAddEdit:Init:btnApply:Click										
TagAddEdit:Init			0.1							

Fig. 4. Emission matrix with correction.

In order to compute the F_{int} score for G_1 , we average all the pairwise similarities from within G_1 .

$$\begin{aligned}
 F_{int}(G_1) &= \frac{O(S_a, S_b) + O(S_a, S_c) + O(S_b, S_c)}{3} \\
 &= \frac{0.786 + 0.627 + 0.658}{3} \\
 &= 0.690
 \end{aligned}$$

Similarly, we compute $F_{int}(G_2)$.

$$\begin{aligned}
 F_{int}(G_2) &= \frac{O(S_d, S_e) + O(S_d, S_f) + O(S_e, S_f)}{3} \\
 &= \frac{0.682 + 0.841 + 0.773}{3} \\
 &= 0.765
 \end{aligned}$$

The F_{ext} score is computed the same way, only using all pairwise similarities from two different installations.

$$F_{ext}(G_1) = \frac{0.423 + 0.400 + 0.128 + \dots}{9}$$

$$F_{ext}(G_1) = 0.320$$

In this case, $F_{ext}(G_2)$ is also 0.320, since the same nine pairwise similarities exist for G_2 . The final fingerprint score for each is the ratio of internal to external score.

$$FP(G_1) = \frac{0.690}{0.320} = 2.156$$

$$FP(G_2) = \frac{0.765}{0.320} = 2.391$$

2.6. Applying to regression testing techniques

Because of how this product is architected, automated regression tests produce the same set of telemetry that user interactions do. So a test session T_y is equivalent to a user session S_j .

$$T_y = S_j = \{a_1, a_2, \dots, a_l\}$$

Given test cases in a test suite, $\mathbb{T} = \{T_1, T_2, \dots, T_k\}$, we propose to calculate the similarity between a test case and installations, i.e., $F_{test} : \mathbb{T} \times \{G_1, G_2, \dots, G_c\} \rightarrow \mathbb{R}$.

The similarity between a test case and an installation is calculated as the average similarity between the test session and all the sessions in an installation.

$$F_{test}(T_y, G_i) = \frac{1}{|G_i|} \sum_{S_k \in G_i} O(T_y, S_k)$$

These test fingerprint scores F_{test} can then be applied in regression testing techniques. In the test prioritization technique, tests are ordered such that the most important tests are executed first. The fingerprint score $F_{test}(T_y, G_i)$ is computed for all regression tests for an installation based on previous usage. The test cases are then ordered by descending fingerprint score so that tests with the highest fingerprint scores are executed first.

3. Experiment 1

In the first part of our study, we wish to determine the effectiveness of telemetry fingerprinting based on product usage. Our hypothesis is that by fingerprinting usage we can improve the results of test case prioritization techniques such as test case prioritization. We believe that including previous usage information (the fingerprint) will allow the prediction of which areas of the product will receive more usage in the future and therefore be more likely to contain discoverable bugs. To investigate this hypothesis, we consider the following four research questions.

In this study, we use the term “environment” to indicate one installation of the software product for one customer. The same version of the software is installed in each case, and the only difference between environments is in the usage of the software. While the set of bugs in the software is the same regardless of environment, each environment has a different set of “discoverable bugs”.

- RQ1: Can fingerprinting be useful for improving test case prioritization techniques?
- RQ2: Are fingerprint scores effective in identifying usage patterns?

- RQ3: Can stable high fingerprint scores be computed for different environments?
- RQ4: Which fingerprinting techniques perform better in terms of test case prioritization?

RQ1 examines the usefulness of fingerprinting in test case prioritization techniques. Specifically, we use fingerprinting to prioritize test cases by environment. We then measure the effectiveness of the prioritization against the discoverable bugs in each environment.

RQ2 attempts to determine if the fingerprint scores are actually detecting usage patterns, or if they are measuring different aspects of the software. To investigate RQ2, we randomize a set of sessions from across multiple environments and then use fingerprint scores to predict the source environment from which the session originated. If we are able to accurately predict the source environment based on fingerprint scores alone, this would indicate that fingerprint scores are effective at identifying usage patterns.

RQ3 deals with the ratio of internal to external scores (the overall fingerprint score). An effective fingerprinting algorithm will yield a higher ratio, while an ineffective algorithm would yield a ratio closer to 1. To verify that the algorithm is effective and not producing random results, we would expect to find high fingerprint scores across multiple environments.

Finally, RQ4 compares different similarity functions used in fingerprinting to determine which algorithms perform the best in test case prioritization. To evaluate this research question, we compute the APFD and total runtime to find all discoverable faults across algorithms.

3.1. Object of analysis

This first study was conducted based on a prerelease version of *Microsoft Dynamics AX*. This software product is delivered in a Software-as-a-Service (SaaS) model and collects telemetry data necessary for monitoring and maintaining the hosted product. Fig. 5 illustrates the pipeline for collecting and reporting product telemetry. The overhead for this telemetry collection has been found to be around 1–3 percent in performance testing.

This product has a browser-based interface. Each time the user performs an action, a *UserInteractionMarker* event is recorded in telemetry. While the specific details of the user interaction are hashed due to privacy concerns, the events provide unique markers that can be used in fingerprinting usage data. A chronologically ordered set of user interactions by the same user is called a “user session”. We chose Dynamics systems because they are early adopters and provide with extra telemetry to be able to investigate our proposed techniques.

Approximately once every five minutes, the collection of *UserInteractionMarker* events are sent to the cloud-based Azure Monitoring and Diagnostic Service (MDS). As part of this transfer, all identifiable user information is scrubbed and hashed out from the events to ensure user privacy. The resulting aggregate information is stored in Azure Hadoop BLOB storage, from which telemetry reports are generated. Because user information has been hashed out, there is no way to determine individual user actions, but usage trends can be derived.

We used telemetry streams from seven prerelease environments in this study. Because worked closely with these customers in validating the prerelease software, we were able to obtain more detailed informa-

Table 2

Session information by dataset.

Dataset	Number of sessions	Average session length
Installation A	500	182
Installation B	500	281
Installation C	500	421
Installation D	500	141
Installation E	500	193
Installation F	500	479
Installation G	500	173

tion than may otherwise be available. We will refer to these environments as “Installation A” through “Installation G”.

We randomly selected 500 user sessions out of each of these environments, with each session containing at least ten user actions. The sessions were all collected at the same time, so there is no seasonal variability in the usage data. Table 2 describes the specific datasets. We do not have access to user information, so some users may have performed only one session while other users may have performed many sessions.

3.2. Variables and measures

In RQ1 and RQ4, the independent variable is the prioritization technique, and the dependent variables are the average percentage of fault detected (APFD) and total runtime of the test suite until all discoverable bugs are found. There is one independent variable in RQ2 and RQ3, the fingerprint algorithm. For RQ2 the dependent variable is the percentage of sessions for which the owner was correctly identified, while in RQ3 the dependent variable is the fingerprint score per environment.

3.3. Experimental process

To investigate RQ1, we wish to see what impact fingerprint techniques would have on test case prioritization. The most common measure of prioritization effectiveness is APFD (average percentage of faults detected) [43]. The APFD value ranges between 0 and 100, and higher numbers imply faster fault detection rates. We also examine the total runtime of the tests necessary to expose all discoverable bugs.

Fig. 6 illustrates the process for RQ1. We start by injecting artificial faults, essentially marking individual interactions from test sessions as having a fault. Artificial faults were necessary, as this is a new product, so faults from updates do not exist yet. These are all the faults that could potentially be found by running tests. We chose to simulate 10% of the interactions being faulty. Similar results are seen with both higher and lower percentages, but 10% yields the most stable results across repeated runs. Smaller percentages yield the same average results, but with more variability.

The user sessions are then split into two groups to simulate what exists before and after a software update is shipped. The set from before the update is used to prioritize the test cases using their fingerprint scores. The second set, from after the update, is used to determine which of the injected faults would have been discovered by users after the update. If a faulty interaction was not invoked by any users, then that fault was non-discoverable.

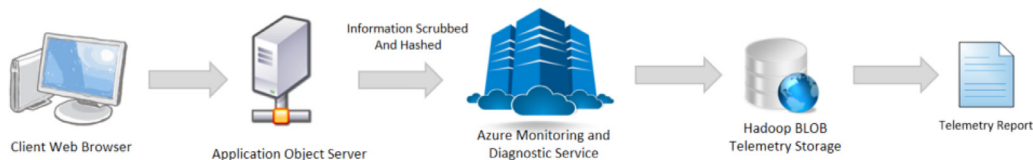


Fig. 5. Telemetry pipeline.

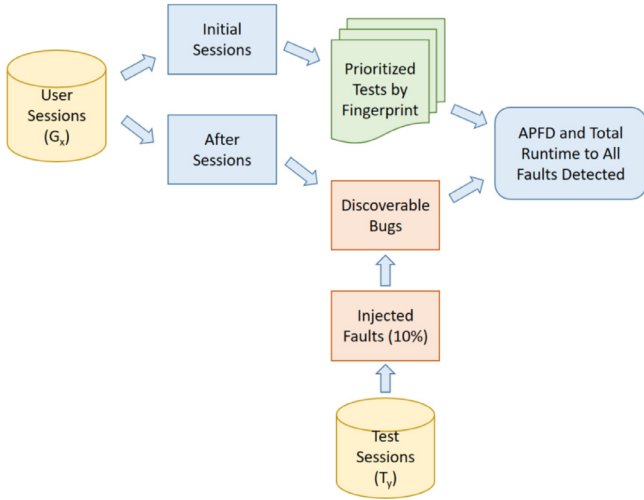


Fig. 6. Experiment process for RQ1.

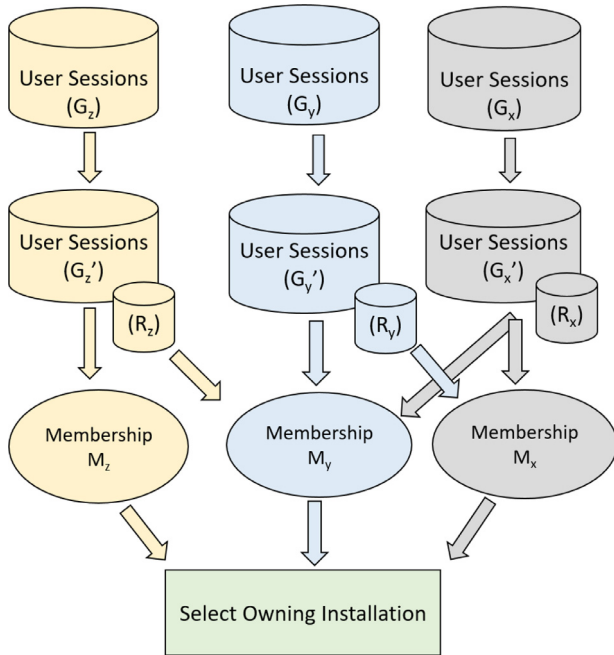


Fig. 7. Experiment process for RQ2.

Finally, we use the prioritization of tests based on fingerprints to compute the APFD for the discoverable bugs. The “Action Match ratio” technique was used in this research question because it was shown to be one of the most effective in RQ4. We also compute the total runtime of

the tests (in terms of number of total interactions) necessary to expose all discoverable bugs, because test execution time varies widely across test cases. By considering the total runtime to uncover all bugs, we have a better understanding of the effectiveness of the prioritization.

The baseline we compare against is “greedy prioritization”, which seeks to prioritize by coverage that is, which test covers the most distinct interactions first. The greedy technique is one of the most widely used baseline prioritization techniques in this area of research [34].

Once the fingerprint scores are known, in RQ2 we then wish to see if fingerprint scores can be used to accurately identify the owning installation of a given session. Fig. 7 shows the process for this step.

For each installation G_i , we randomly select 10% of the sessions and add them to R_i . Other values above and below 10 percent were also tried and yield similar results. The selected sessions are removed from the initial data set, leaving G_i' . We then compare each of the randomly selected sessions with all the sessions in each installation's G_i' to compute the membership score M_i for each. Whichever installation yields the highest membership score is selected as the owning installation. From that we compute the percentage of sessions that were correctly mapped to their original installation.

Fig. 8 shows the process used to determine fingerprint scores for each similarity algorithm to evaluate RQ3. The sessions for each installation G_i are used to compute the internal and external fingerprint scores, from which the final $FP(G_i)$ for that installation can be computed.

Finally, to evaluate RQ4 we again use the same experimental process illustrated in Fig. 6. We inject faults and compute the APFD and Total Runtime to All Faults Detected. In this case we compute these values across all of the similarity functions previously discussed. The best algorithms will yield a high APFD together with a low total runtime to find all discoverable faults.

3.4. Data and analysis

In this section, we present the results of this study as described in the previous sections.

3.4.1. RQ1 Analysis

RQ1 investigates the applicability of usage fingerprinting in the area of test case prioritization. As previously described, we introduced injected faults in the interactions that tests invoke and then determined which of those faults would be discoverable by at least one user after the update. We then computed the APFD and total test runtime in terms of the number of interactions necessary to expose all discoverable faults. Figs. 9 and 10 show the results. The bar graphs in Fig. 9 show the APFD, and the line plots in Fig. 10 indicate the test runtime. We included one additional installation in the results which became available after the research on RQ2 and RQ3 was completed.

To illustrate the results variation of APFD, and time across different installations, we also show the results in the boxplots, which is shown in Figs. 11 and 12.

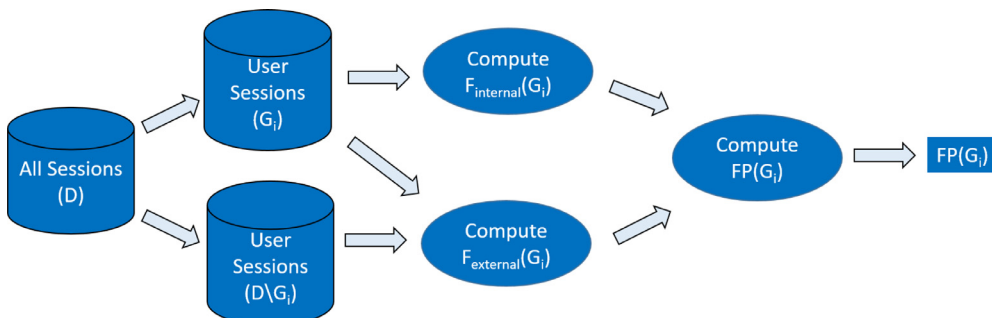


Fig. 8. Experiment process for RQ3.

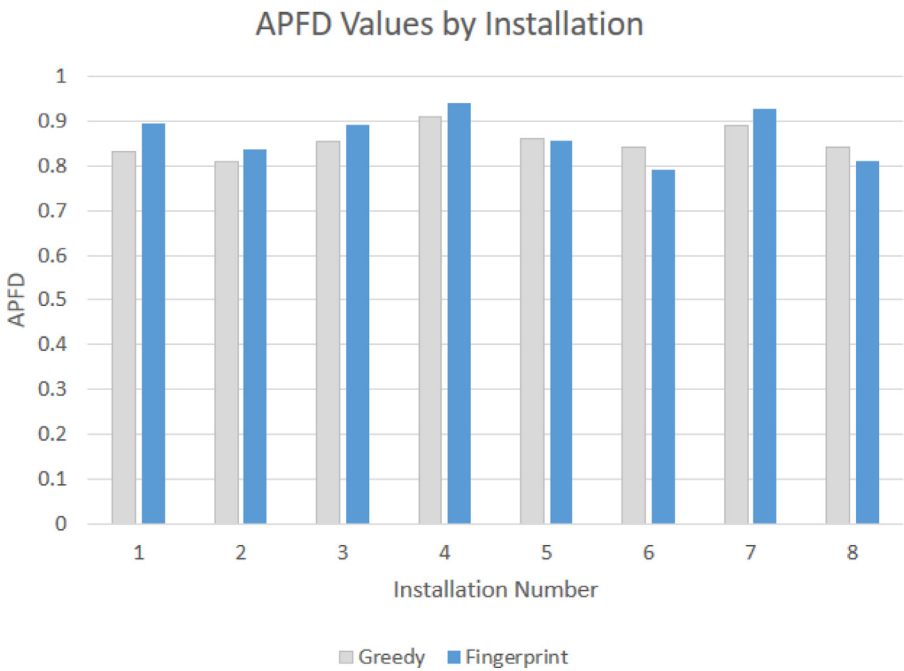


Fig. 9. APFD values by installation.

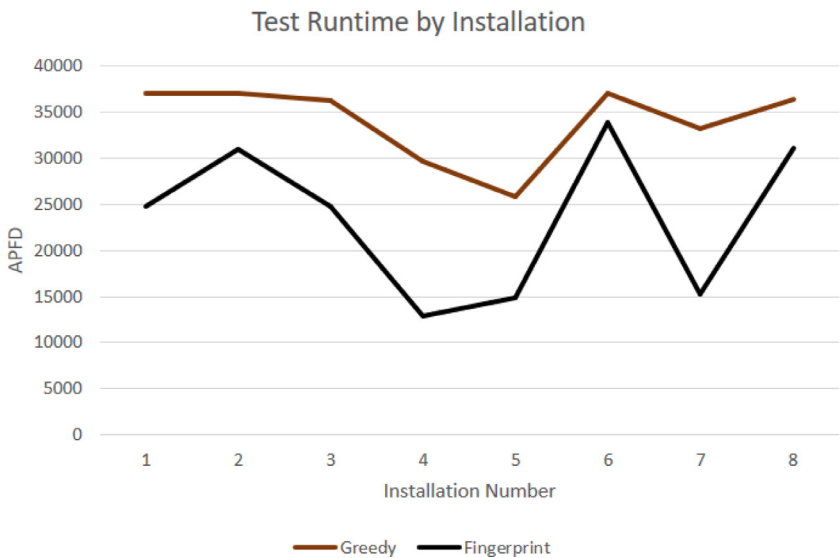


Fig. 10. Test runtime by installation.

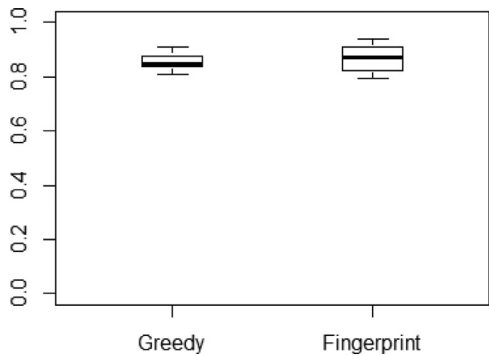


Fig. 11. APFD values variation.

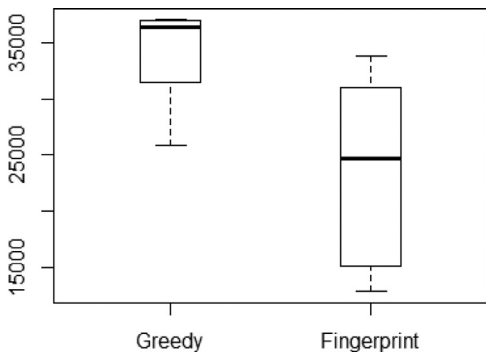


Fig. 12. Runtime variation.

For this experiment, 1483 tests were used. Those 1483 tests used a total of 10,610 distinct interactions; thus, 1061 were randomly selected as faulty. This experiment was performed 20 times with random selections each time and the results shown are the average across all

runs. The 1061 faulty interactions were then intersected with the interactions from the “after sessions” for each installation, yielding on average between 20 and 60 discoverable faults in each case. APFD values were computed by prioritizing all 1483 test cases based on those 20–60

discoverable faults. The total runtime was computed based on the total sum of interactions in the prioritized test cases until all discoverable faults were exercised. Both thresholds higher and lower were also tried, and they yielded similar results. The 10% figure was selected as it provided sufficient number of discoverable faults to ensure that APFD values were fairly stable between experiment runs, with a standard deviation of less than 5% across all installations.

In five of the eight installations, prioritization by fingerprint yielded a higher APFD than greedy prioritization. However, all the APFD values were within a few percentage point of each other. While the average APFD was around 1.5% higher for prioritization by fingerprint, it seems that the effectiveness of fingerprint prioritization is roughly equivalent to greedy prioritization in terms of APFD.

A more interesting result was achieved by the total test runtime by prioritization technique. In all installations, fingerprint prioritization yielded a much smaller runtime to expose all discoverable faults than greedy prioritization. This is because greedy prioritization orders the tests merely by putting those with the most unique interactions first. Those same tests are usually the longest running tests as well. Fingerprint prioritization however focuses on those tests that are actually more likely to contain discoverable faults. On average, the total test runtime was reduced by more than 30% using fingerprint prioritization.

We believe this is a deficiency in the APFD scoring mechanism in that it treats all faults as equal, while this is not the case in the eyes of a customer. To a customer, the fault which is currently blocking their work is the most critical fault. It is that additional dimension of customer impact which this research is adding to the equation. Additionally, APFD does not consider total test runtime. Test runtime is an important measure since the reason for using test case prioritization in the first place is to reduce runtime necessary to find faults. APFD is one measure of efficiency in the engineering process in terms of what quality can be delivered to customers. The customer impact of a fault in terms of discoverability is another important dimension of efficiency. The reason we still compare against APFD is we need to show this new technique still measures up against more traditional techniques and measures.

To determine whether the differences observed between the proposed technique and code coverage based technique are statistically significant, we performed the Wilcoxon Rank-Sum test considering the average APFD scores for each technique, at a significance level of 0.05. The result (p -value = 0.003569) shows that the fingerprinting technique is significantly outperformed the greedy technique.

3.4.2. RQ2 analysis

In RQ2, we wish to see if the fingerprint scores discussed in RQ1 have any ability to accurately capture usage patterns. To test this, we randomly took 10% of all sessions from each installation and held them as a pool of 350 sessions from unknown installations. We then computed the membership score M_i for each with the remaining 450 sessions in each installation. Whichever installation yielded the highest membership score was selected as the owner. Since we know which installation each session actually came from, this lets us compare the predicted membership to the actual.

Fig. 13 shows the results of this exercise. As expected, we find that session length and E-Count techniques were very poor predictors of membership. Random chance yields a 1 in 7 (14%) chance of accurate membership selection. Form Match and Action Match, on the other hand, were very effective, steadily yielding around 80% accurate predictions across all installations. From this, we can respond “yes” to RQ2: the use of fingerprint scores *does* accurately capture the usage characteristics of an installation.

We like to add one final note of interest. As discussed in the previous section, the Markov Chains technique yielded by far the highest fingerprint scores. However, the accuracy of the prediction of the Markov Chains technique was lower than that of the other methods, hovering around 60%. While this is good, it is worse than two other methods that had lower fingerprint scores. After manual analysis of the raw data, we found that this result can be explained by the high variation in the Markov scores. When the Markov Chains technique properly matches two sessions, it yields a very high internal similarity together with very low external similarities. This artificially inflates the fingerprint score. However, for a large portion of the sessions, the Markov Chains technique yields low scores both internally and externally. When averaging

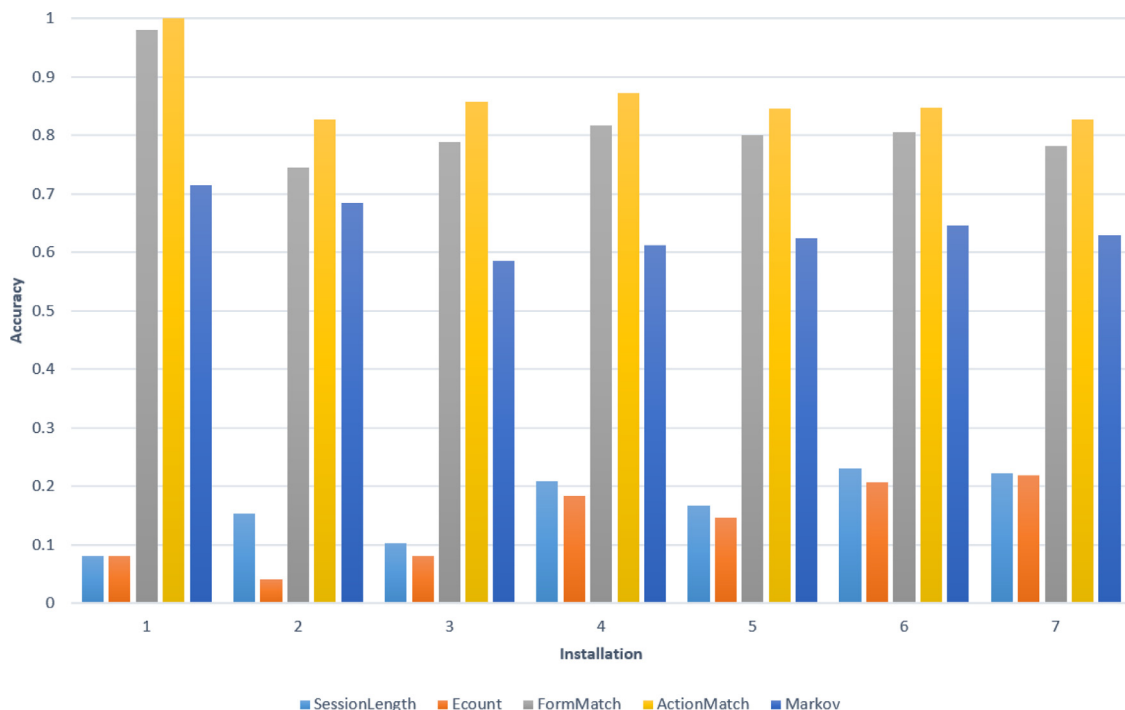


Fig. 13. Accuracy of owner installation prediction.

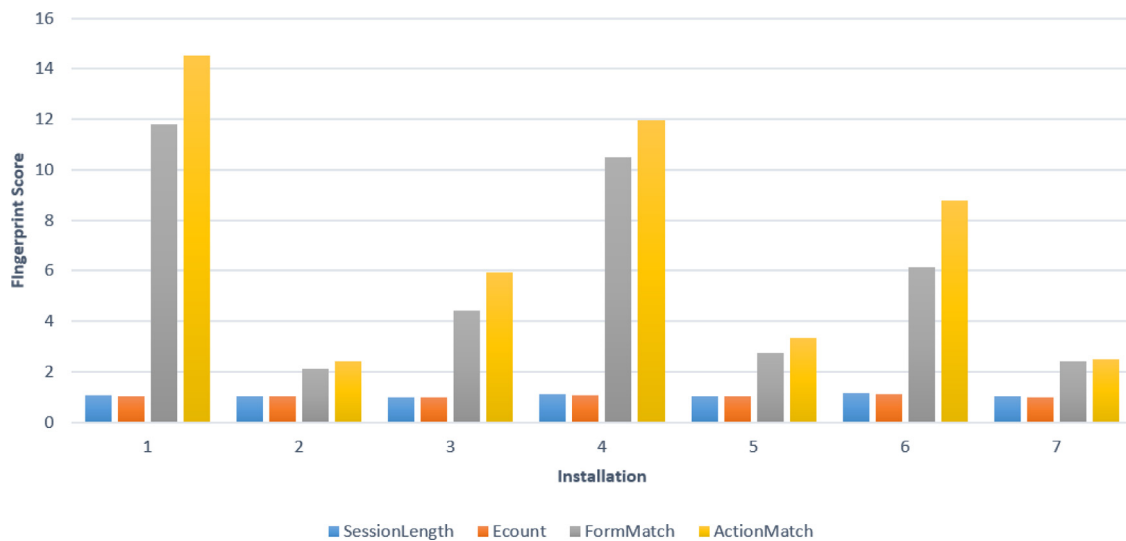


Fig. 14. Fingerprint score by installation and method.

fingerprint scores, those small numbers of very high fingerprint scores artificially inflate the average.

3.4.3. RQ3 analysis

RQ3 seeks to find algorithms for which there is a high internal to external ratio, yielding a high fingerprint score. A purely random algorithm with no measure of similarity between sessions would yield a fingerprint score (ratio) averaging 1 across repeated experiments. There is an upper bound on fingerprint score as well, since not all sessions from the same installation are identical. Theoretically, the upper bound approaches infinity as internal scores approach 1 and external scores approach 0. Pragmatically, however, this is not possible, unless internal sessions are nearly identical and external sessions are very different. In practice, because customers are all using the same software, there will be an upper limit on any fingerprint score. We do not have a quantification of that limit at this time.

Fig. 14 shows the fingerprint score by installation and similarity method. As expected, naïve approaches such as comparing by session length or the number of “E” characters in names yield fingerprint scores very close to 1. The methods of ratio of Form Match and Action Match are much better, yielding fingerprint scores up to 15.

Also note that the relative effectiveness of each fingerprint method remains relatively constant across installations. Some installations such as “Installation 1” have higher overall values as they are unique in their usage and therefore easier to fingerprint. “Installation 2” and “Installation 7” in this case are not very unique, yet they still exhibit fingerprint scores close to 3, meaning they are three times more internally similar than they are similar across installations.

One method not shown in this graph is similarity by Markov Chains. That is because the graph’s scores are very skewed, having a fingerprint score up to 10,480 for “Installation 3”. While the fingerprint score is very high, the internal similarity is very low (less than 0.0001 in most cases). It is only because the cross-session similarity is orders magnitude smaller, while the scores are so large. While this may seem like a very successful result, as we discuss in the next section, there are some drawbacks.

3.4.4. RQ4 analysis

In RQ4, we seek to find which algorithm for a similarity function yields the best performance for test case prioritization. There are two measures tracked here, APFD and average total runtime to find all discoverable faults.

Table 3

APFD Scores and total test execution time.

Similarity measure	APFD	AVG runtime
Session Length	0.805	34,032
Action Match	0.870	24,559
E-Count	0.743	34,415
Form Match	0.882	24,386
Markov Chain	0.467	33,755

Table 4

Action match and form match average APFD by installation.

Installation	Action Match APFD	Form Match APFD
Installation A	0.891	0.916
Installation B	0.848	0.849
Installation C	0.880	0.902
Installation D	0.938	0.947
Installation E	0.892	0.874
Installation F	0.779	0.805
Installation G	0.917	0.931
Installation H	0.812	0.835

From the results in Table 3, the Action Match and Form Match algorithms are clearly the most efficient, both in APFD as well as total runtime. Form Match was slightly more efficient at 0.882 APFD compared to 0.870 APFD from Action Match. This difference is small enough to consider both to be roughly equivalent.

As previously discussed, similarity functions such as E-Count encode some information about the actions being performed, but not very precisely. But these techniques are not as good as ones which encode more information.

The Markov Chain technique performed very poorly, having an APFD worse than 0.5. As previously discussed, this appears to be an overfit of the data causing poor performance. This result demonstrates the importance of testing various approaches to ensure the algorithm being used is actually beneficial.

Table 4 shows a breakdown of average APFD by installation. This shows that the techniques vary in effectiveness across installations, as well as in their comparative effectiveness. For instance, in Installation A, the Form Match algorithm yields a higher APFD, while in Installation E the Action Match algorithm has a higher APFD.

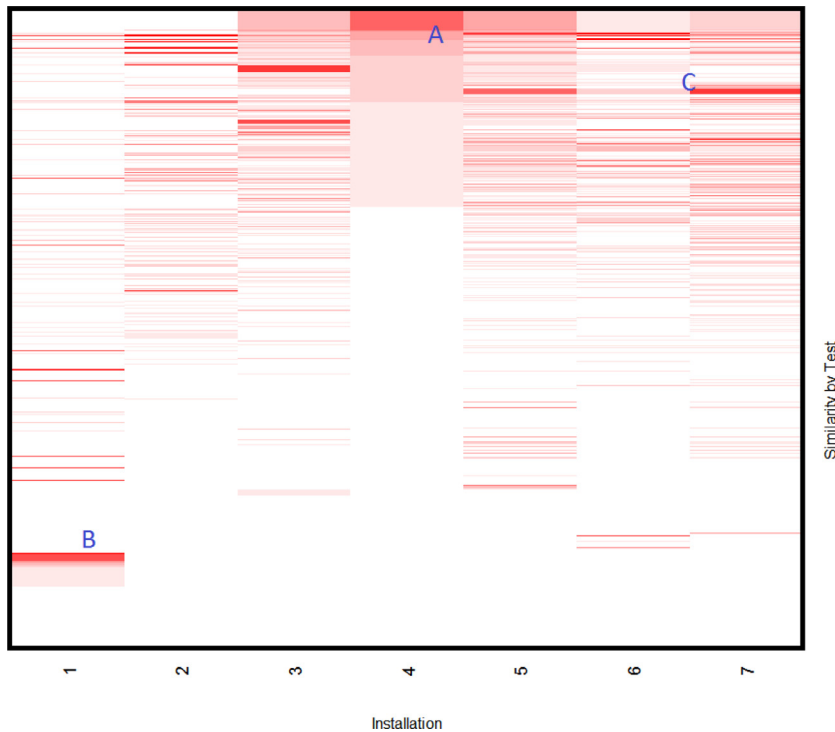


Fig. 15. Test similarity by installation. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

3.4.5. Additional analysis

To further understand why the results from RQ1, RQ2, RQ3, and RQ4 occurred, we performed some additional analysis of the data. This analysis is not a research question itself, but rather qualitative analysis of the data to help understand the results. This additional analysis is not replicated in other experiments which yield similar results.

Fig. 15 shows a heat map of the fingerprint score of each test by installation. The rows represent the individual tests used in this study. The columns represent the seven installations. White indicates no similarity between the test and the installation usage, while red indicates high similarity.

A few interesting details stand out in this heat map. First, the band labeled “A” shows a group of approximately 130 tests that have high similarity to Installation 4. It is interesting to note that these same tests also have high similarities to Installations 3, 5, 7, and to lesser

degree 6. The implication is that these tests cover common functionality used by multiple customers.

The band labeled “B” shows another notable pattern. The roughly 100 tests in this group have high similarity to Installation 1 but virtually no similarity to any other installation. Finally, the band marked “C” illustrates a group of around 20 tests with high similarity to Installations 7 and 5 but little similarity to the other installations.

From this heat map, we see that the intersection between usage patterns varies widely between installations. Because of this, customized prioritization for each installation will yield very different test orderings, allowing a greater number of similar tests to be run far earlier for any given installation.

We next investigated what the impact on test case selection would be by using fingerprinting. Fig. 16 shows the normalized fingerprint score of each test in descending order for each installation.

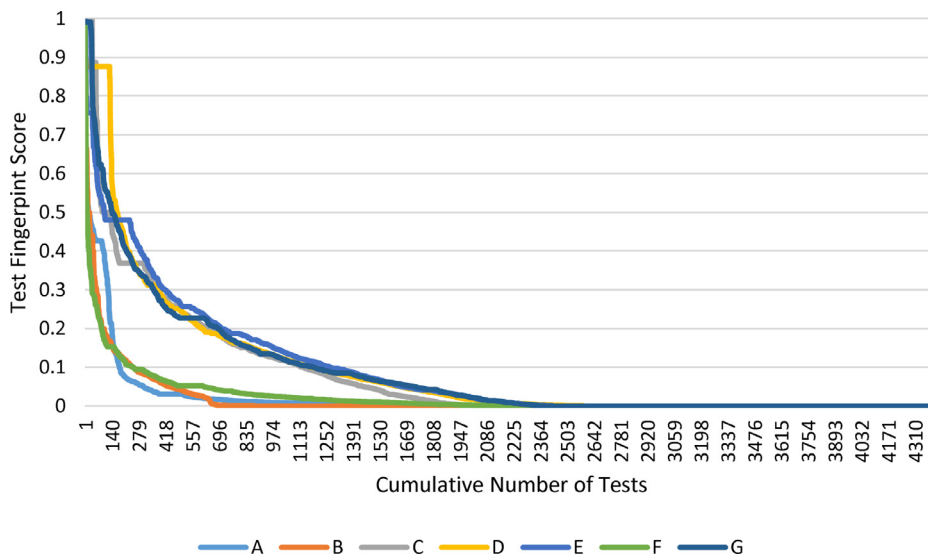


Fig. 16. Test fingerprint scores.

As we can see, the fingerprint scores drop off significantly within the first 10% of tests. Given constrained testing resources, this allows for the execution of all the most important tests while saving 90% of execution time. We also see that there are two groups of lines, one dropping off significantly faster because three of the installations use only a single module each. The other installations all use multiple modules, leading to more gradual decline in scores.

Even if the cutoff threshold were set at a fingerprint score of 0 (meaning to select any test that has any similarity at all to the installation), only 60% of the tests would be selected. Another way of looking at this is 40% of the full regression test suite is testing functionality that is not utilized in any way by any of these seven installations. Based on this analysis, we feel that fingerprinting promises to be an extremely effective method of prioritizing and selecting tests.

4. Experiment 2: a replicated study

In the first study, we showed that the use of telemetry data for test prioritization is effective in terms of test runtime and fault detection rate. Although we investigated research questions using a large industrial application, we considered only one application, which might compromise the external validity of our study. To reduce this threat to validity, we replicated the first study using two other applications. The research questions in this experiment are the same as those in Experiment 1, and for completeness we repeat them here, but we relabel them RQ1', RQ2', RQ3', and RQ4'.

- RQ1': Can fingerprinting be useful for improving test case prioritization techniques?
- RQ2': Are fingerprint scores effective in identifying usage patterns?
- RQ3': Can stable high fingerprint scores be computed for different installations?
- RQ4': Which fingerprinting techniques perform better in terms of test case prioritization?

4.1. Objects of analysis

For this experiment, we used two web applications: *CarrotCakeCMS* and *DASCP*. *CarrotCakeCMS* is an open source content management system [1]. *CarrotCakeCMS* is written in Asp.Net C#, and it uses an SQL server as a database. The main function of *CarrotCakeCMS* is to create a customized website without coding. This application helps users build their websites with little knowledge of website design. *DASCP* an abbreviation for Digital Archive and Scan for Civil Projects, is a proprietary software product that we obtained from a private company. This application is a web based application designed to store civil project contracts including technical information for civil and construction projects such as project plans and other relevant information. *DASCP* includes an access control system, and it provides two types of access rights: one user group has permission to edit or insert project information or upload maps and contract sheets, while the other user group is only allowed to view the data and details about the projects. We chose these two systems for the following reasons: *CarrotCake CMS* is an open source and easy-to-use web application that would be suitable for nonprofessional users; *DASCP* facilitates coherent workflows for user tasks and its user sessions are available.

In order to collect the user session data for these applications, we implemented a small function to store user sessions. We consider a sequence of each user's interactions on a specific date as a user session.

Table 5 lists associated data for each of our objects of analysis: "Classes" (the number of classes), "LOC" (lines of code), "Sessions" (the number of sessions), "Faults" (the number of seeded faults), "Version" (the program versions that we used for the experiment), "Test Cases" (the number of test cases), and "Installations" (the number of places where we installed our applications).

Table 5
Experiment objects and associated data.

Metrics	CarrotCakeCMS	DASCP
Classes	84	107
LOC	20,073	35,122
Sessions	81	748
Faults	20	35
Versions	1	3
Test cases	70	95
Installations	4	3

4.2. Variables and measures

For RQ1' and RQ4', the independent variable is the test case prioritization technique. The dependent variables are the average percentage of faults detected (APFD) and test execution time for catching all discoverable faults. For RQ2' and RQ3', the independent variable is fingerprinting technique. The dependent variable for RQ3' is fingerprinting score, and for RQ2' the dependent variable is session owner prediction accuracy.

4.3. Experimental setup

In this section, we will describe the process of data collection and calculation.

4.3.1. Data collection

The source code of *CarrotCakeCMS* is publicly available, but the web application is not publicly hosted. In order to run *CarrotCakeCMS*, we installed the software in four different locations at the University of North Texas, and we assigned the same installation ID to each group of students in each specific location. The server has a Core i7 CPU and 16 gigabytes of RAM. After uploading *CarrotCakeCMS* on the server, we asked student volunteers to work with the application for 20 days. We had 20 users in altogether, and we divided them into two different role groups to collect different usage patterns depending on role: administrator and end users. Administrator had full access to the whole system, and end users could access almost all components of the system except the access role and membership components. In total, 81 user sessions were collected during the designated 20-days period.

The data collection process for *DASCP* differed from that of *CarrotCakeCMS*. *DASCP* appears in three versions, each of which has been installed on the server of three local construction companies, respectively. The users of *DASCP* from whom we collected the data are real users, and they have application domain knowledge. We collected twelve-months period of user interactions for *DASCP*. As described in Section 4.1, *DASCP* provides two types of access rights for users. In this study, we only considered the sessions of users who have full access to the system.

4.4. Data and analysis

In this section, we present the results of our second experiment and data analysis, for each of our research questions in turn.

4.4.1. RQ1' analysis

RQ1' investigates whether fingerprinting can improve the effectiveness of test case prioritization. Fig. 17 shows the APFD scores and total test execution times for four different installations of *CarrotCakeCMS*. In this experiment, the total test execution time refers to the time taken to expose all discoverable faults. The bar graphs show the APFD scores, and the line plots indicate the test runtimes. In this experiment, we seeded 20 faults, and then we ran test cases to see how many of those faults were detected during user sessions (Fig. 18).

Figs. 19 and 20 show the variation of APFD and runtime for *CarrotCakeCMS* across different installations.

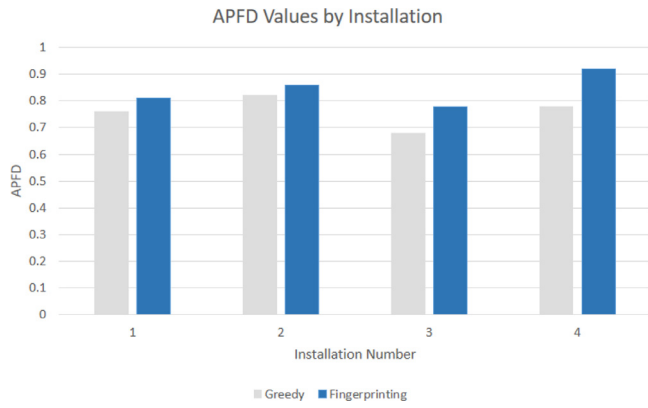


Fig. 17. APFD scores of CarrotCakeCMS for four different installations.

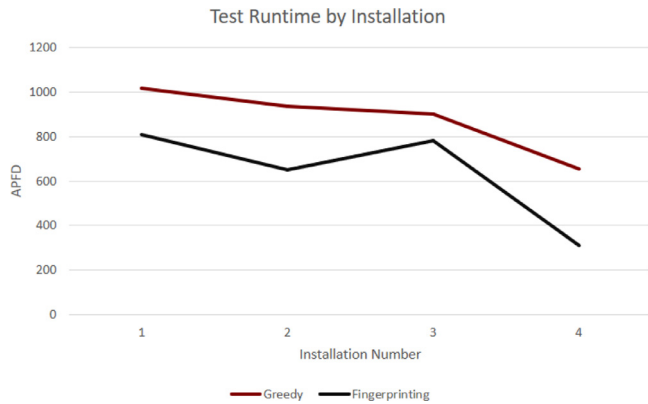


Fig. 18. Runtime of CarrotCakeCMS for Four different installations.

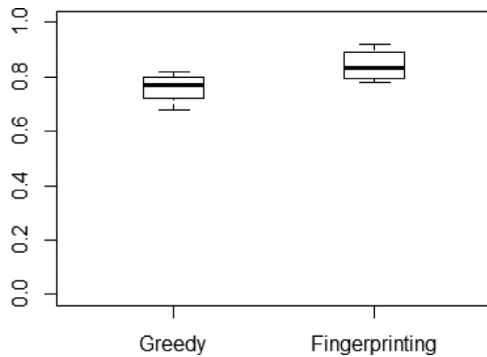


Fig. 19. CarrotCakeCMS APFD values variation.

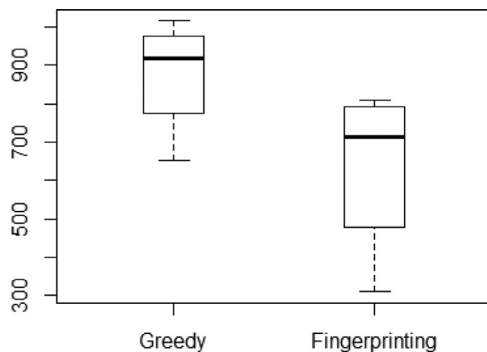


Fig. 20. CarrotCakeCMS runtime variation.

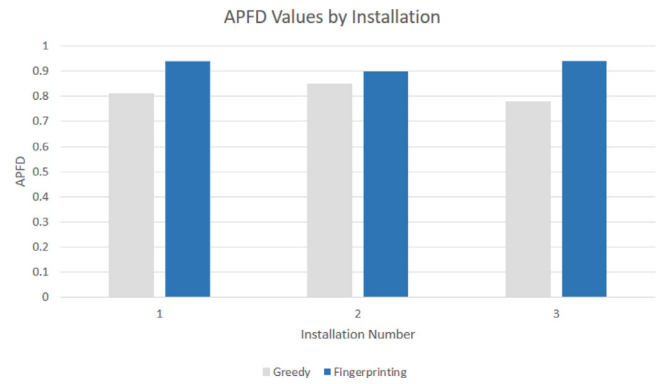


Fig. 21. APFD Scores of DASCP for three different installations.

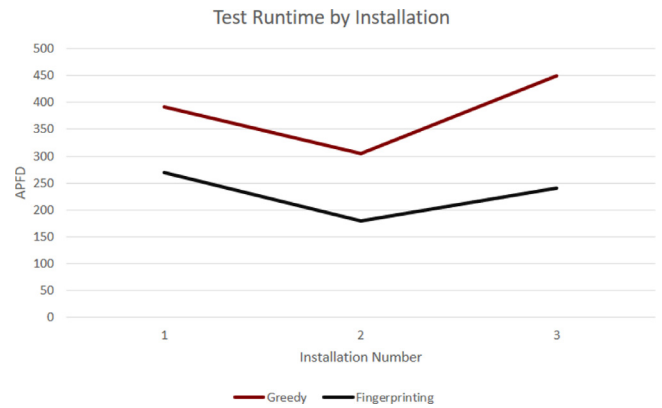


Fig. 22. Runtime of DASCP for three different installations.

Among the five aforementioned fingerprint techniques, we chose Action Match to compare with a greedy technique that has been used widely. As shown in the figure, the fingerprinting technique produced higher APFD values and lower test execution times for all four installations. The average total runtime to find all discoverable faults in *CarrotCakeCMS* using the fingerprint approach is 637 s, while runtime average 878 s using the greedy approach.

Fig. 21 shows the APFD scores and total test execution times for three different installations of *DASCP*. We used the same technique in this experiment that we did for *CarrotCakeCMS*; also, the variables and measures are the same. And similar to the results for *CarrotCakeCMS*, the fingerprint technique produced higher APFD values and lower test execution times for all three installations. In this study, we seeded 35 faults. All discoverable faults in installation 1 of *DASCP* were found by running only 34.7% of the reordered test cases with the fingerprint technique, but, using the greedy technique, 57.8% of the test cases were required to detect all such defects. For Installations 2 and 3, all discoverable faults were detected by executing 29.4% and 41.05% of test cases reordered by the fingerprint technique, respectively. However, the greedy technique required more test cases and took more time to detect all discoverable faults. We had to run 51.5% of test cases taking 305 s of execution time for Installation 2, and 60% of test cases taking 450 s of execution time for Installation 3. Further, for the fingerprint technique, the average test execution time for three installations was 229 s, but for the greedy technique it was 382 s, or 1.7 times slower than the fingerprint technique (Fig. 22).

Figs. 23 and 24 show the variation of APFD and runtime for *DASCP* across different installations.

To determine whether the differences observed between the proposed technique and code coverage based technique are statistically significant we performed non parametric statistical analyses, considering

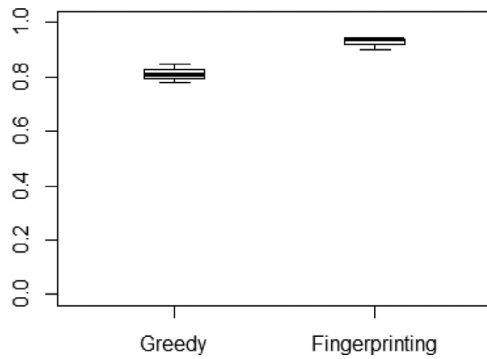


Fig. 23. DSCP APFD values variation.

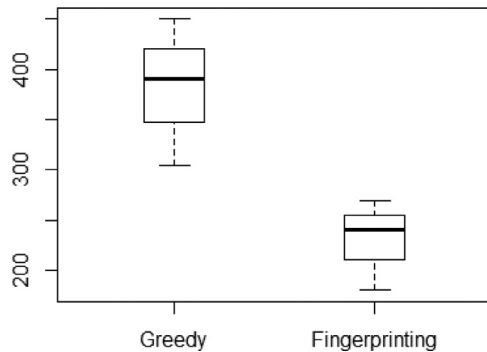


Fig. 24. DSCP runtime variation.

Table 6
Wilcoxon Rank-Sum test, fingerprinting vs greedy technique.

Project	CarrotCakeCMS	DASCP
p-value	0.004545	0.000621

the average APFD scores for both *CarrotCakeCMS* and *DASCP*. Table 6 presents the results of analysis, for a significance level of 0.05. The Wilcoxon Rank-Sum test reports that the fingerprinting technique is significantly outperformed the greedy technique.

4.4.2. RQ2' analysis

RQ2' considers whether fingerprint scores are effective in identifying user interaction patterns. To investigate RQ2', we followed the same data collection process that were used for RQ2 in Experiment 1, with a minor change. For *CarrotCakeCMS*, instead of selecting 10% of the data from each installation, we choose 30% because the total number of sessions was very low.

Fig. 25 presents the results for *CarrotCakeCMS*. The vertical axis shows the accuracy of prediction, and the horizontal axis shows the installation number. Initially, we had four installations for *CarrotCakeCMS*, but one of the installations had only one user, so we eliminated that installation. The Session Length technique produced the lowest value result, while the Action Match and Form Match produced relatively higher values than other techniques.

By looking at our raw data, we found that these results were caused by unplanned users tasks. We did not define a procedure of tasks for our users and simply just asked them to create their own custom websites. For example, one user performed only 1 task that returned a session length of 2 on a certain day, while another user on the same day performed multiple tasks that returned a session length of 38.

Fig. 26 shows the results for *DASCP*. In *DASCP*, we had data from more than 700 sessions for three installations, and because this was a sufficient number of sessions to investigate our research question, we ended up using only 10% of the sessions from each installation. As shown in Fig. 26, Action Match and Form Match have the highest scores, and Markov Chain returned higher accuracy than in the *CarrotCakeCMS* experiment.

From the results of this research question, we learned that when the diversity of performed tasks is low, the accuracy of session owner prediction will decrease. For example, in *CarrotCakeCMS*, the majority of tasks performed were creating web pages and updating web template, which are similar tasks. For this application, we obtained the low accuracy (in best case 38% for installation 3 by Form Match technique). However, in the case of *DASCP*, the system was installed in three different companies and each of them had their own workflow. Although most of the performed tasks were similar, the sequences of user interactions were different. As can be seen from Figs. 26 and 25 the accuracy scores for *DASCP* and *CarrotCakeCMS* are quite different. For example, Form Match technique on installation 3 of *DASCP* produced 92%, while Form Match technique on installation 3 of *CarrotCakeCMS* produced 38%.

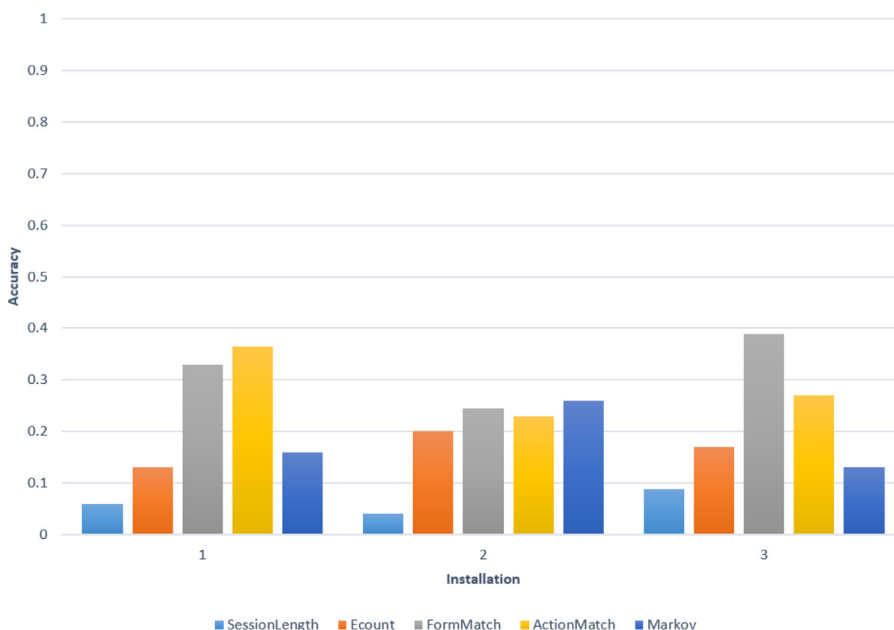


Fig. 25. Accuracy of correct installation prediction for CarrotCakeCMS.

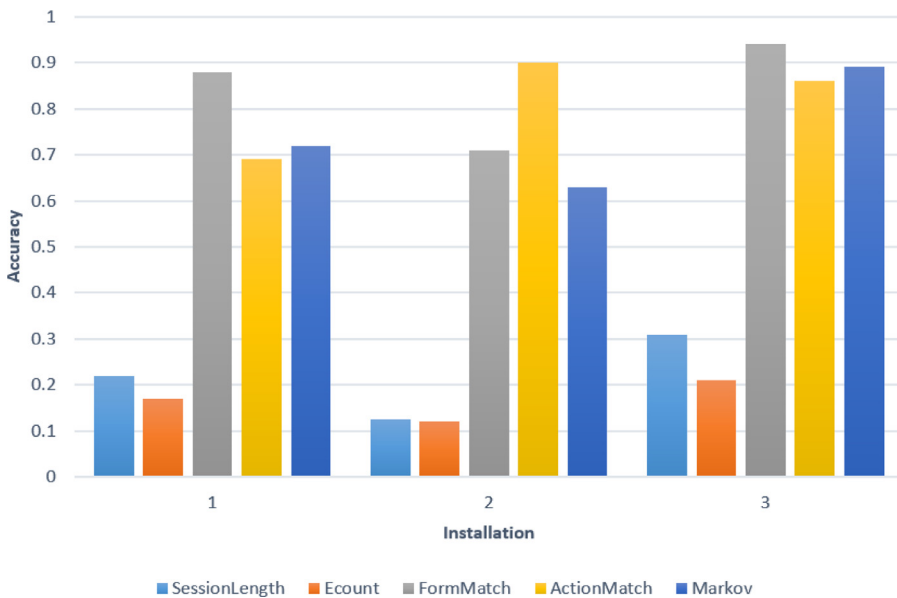


Fig. 26. Accuracy of correct installation prediction for DASCPC.

4.4.3. RQ3' analysis

In RQ3', we investigated whether the fingerprint technique is able to return stable results for different installations.

In order to evaluate RQ3', we applied the same approach as in Experiment 1, with a minor modification. We used the same algorithms used in Experiment 1 to calculate fingerprinting scores for both applications, DASCPC and CarrotCakeCMS.

In Tables 7 and 8, we present fingerprint scores for our applications. The columns in both these tables presents the same metrics. "Technique name" lists the type of fingerprint technique, "AVG FP" indicates the average fingerprint score, and "Std Dev" indicates the standard deviation of the fingerprint scores.

Table 7 shows the fingerprint scores for CarrotCakeCMS. The first three methods ("Session Length," "E-Count," and "Markov Chain") produced low fingerprint scores, and "Session Length" method produced many different patterns, which means, in some cases, that we obtained sessions by length 1 and for other cases by length 48.

As Table 7 shows, Action Match and Form Name methods produced very similar results, while "Markov Chain" produced a much lower score. We examined our database more closely to obtain a better understanding of our results, and we found that disordered users' interactions produced such results. Further, our emission matrix for the Markov Chain technique is very sparse; we could not obtain accurate patterns using this technique.

Table 7
Fingerprinting scores for CarrotCakeCMS.

Technique	AVG FP	Std Dev
Session Length	1.159	0.872
E-Count	0.832	0.103
Markov Chain	0.0681	0.941
Form Match	2.785	0.446
Action Match	2.134	0.287

Table 8
Fingerprinting scores for DASCPC.

Technique	AVG FP	Std Dev
Session Length	3.736	0.127
E-Count	1.54	0.105
Markov Chain	7.452	0.439
Form Match	6.052	0.203
Action Match	8.113	0.415

By examining the raw data in our dataset, we found that the rates of sequence and cohesion between interactions are low. We did not train our users to work with the CarrotCakeCMS system before conducting our experiment and most of them were not familiar with web design techniques; they crawled the system and created forms with random values. Users almost jumped from one page to another and the behavior of most was random. Some 65% of users made use of 46% of the whole web pages, and 90% of users utilized the "UserGroupAddEdit:Add()" and "SiteTemplateUpdate:Update()" functions at least once.

Table 8 shows fingerprint scores for DASCPC. Although we collected sessions from all three companies where DASCPC is installed, because almost all users pursue the same workflow, we obtained high fingerprinting scores. Only 8% of the whole web pages and 17% of the available actions have been never used by any of the software's users. However, the length of the sessions varied by date and workplace.

As we expected, the Markov Chain method produced a high score compared to CarrotCakeCMS. DASCPC users are well trained and familiar with system workflow, and as a result we could extract more accurate patterns. Our emission matrix for DASCPC is not sparse; in contrast, we obtained a number of sequential patterns through our matrix. Almost 92% of the application web forms were used by administrator users, and users performed similar behavior during the time that we evaluated their interactions.

From these results, we learned that when users are actual users and trained, we obtained more accurate results. Compared to CarrotCakeCMS, DASCPC whose users are trained and are familiar with the system workflow, produced higher fingerprint scores. Further, when users are following the intended system workflow, Markov Chain is almost as effective as Action Match technique, which is the best technique among all examined techniques for both applications.

4.4.4. RQ4' analysis

RQ4' investigates which algorithm for a similarity function yields the best performance for test case prioritization. Tables 9 and 10 show the APFD scores and average test execution times for all installations of CarrotCakeCMS and DASCPC. The left column in the tables shows the APFD scores and the right column shows the total test execution time to catch all discoverable faults.

In the case of CarrotCakeCMS (Table 9), Action Match and Form Match outperformed other techniques, and Markov Chain performed worst. In terms of APFD, Form Match (0.861) was slightly better than Action Match (0.845), but their difference is negligible (0.016). In the case of DASCPC (Table 10), Action Match and Markov Chain performed

Table 9
APFD scores and total test execution time for CarrotCakeCMS.

Similarity measure	APFD	AVG runtime
Session Length	0.789	932.562
Action Match	0.845	630.241
E-Count	0.663	1015.386
Form Match	0.861	601.821
Markov Chain	0.537	1365.102

Table 10
APFD scores and total test execution time for DASC.

Similarity measure	APFD	AVG runtime
Session Length	0.793	401.761
Action Match	0.934	233.416
E-Count	0.704	436.473
Form Match	0.871	292.186
Markov Chain	0.912	265.769

better than other similarity functions. Unlike *CarrotCakeCMS*, we can see that Markov Chain in *DASC* produced good results for both APFD and runtime, which are even higher than Form Match. The E-Count technique performed the worse (by 0.704 APFD score).

From these results, we can infer that, Action Match and Form Match are more stable compared to other similarity functions and their results are steady even when the users are not expert and usage pattern are not very coherent. Also, when we have expert users who are familiar with the system workflow, Markov Chain would be a good alternative for Action Match or Form Match.

4.4.5. Additional analysis'

Similar to Experiment 1, we performed additional analyses of the data. A heat map of the fingerprint score of each test case of *CarrotCakeCMS* by its installations is shown in Fig. 27. The figure is structured in a similar way to Fig. 15 in Experiment 1. Band “A” shows a group of approximately 6 tests that have high similarity in all 4 Installations. We examined these 6 test cases and found that they exercise a set of modules such as “SiteTemplate” and “UserGroup”, which almost all users have used. There are few other groups of the test cases with medium similarity, however, these test cases are scattered in the entire heat map

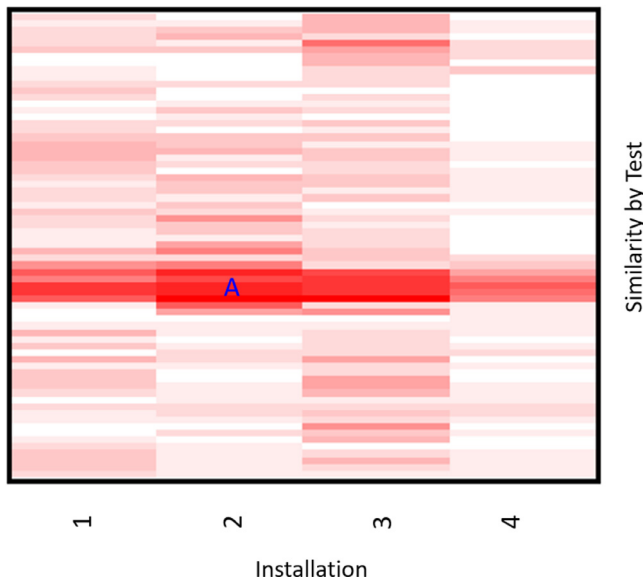


Fig. 27. Test similarity by installation for CarrotCakeCMS.

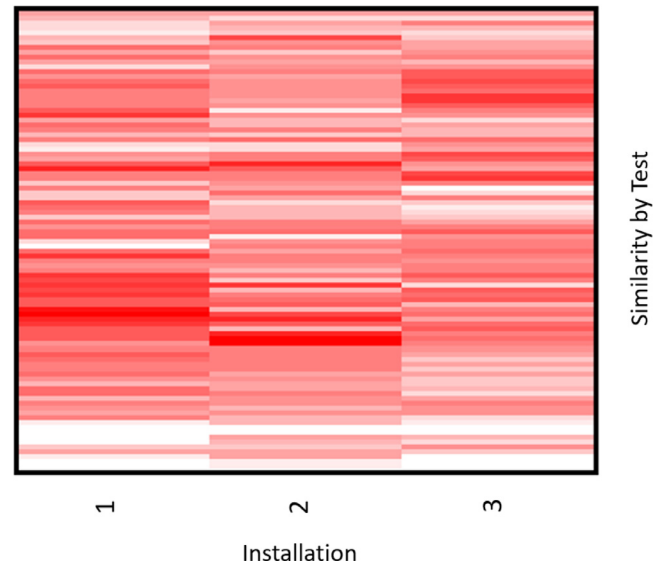


Fig. 28. Test similarity by installation for DASC.

and we could not group them as a set of test cases with high similarity pattern, which could be the result of irregular and sparse user sessions on this application.

Fig. 28 shows a heap map for *DASC*. It is apparent that there is a dense distribution of high similarity scores among all three installations. Unlike Figs. 15 and 27, we could not label a group of test case as a band because the test cases with high similarity score are scattered in the heat map. Thus, from this figure, we could not find noticeable trends. The reason for this is that almost all functions of the applications have been used by users.

Figs. 29 and 30 show the normalized fingerprint score of each test in descending order for each installation in *CarrotCakeCMS* and *DASC*. The goal of this analysis is to investigate whether fingerprinting technique is applicable for test case selection or not. From Fig. 29, we can see that the fingerprint scores drop off significantly after almost 10% of the tests. Fig. 30 also shows that the scores drop off within the first 7% of the tests. Although the number of test cases used in this experiment is much smaller than one in Experiment 1, the patterns of fingerprinting scores for the two experiments are similar. From these results, we can say that even if we set the cutoff threshold for selecting any test case that has a similarity at all to the installation, only 68 and 88% of the entire test suite of *CarrotCakeCMS* and *DASC*, respectively, would be enough to exercise all portion of the systems that have been used by any user in all installations.

5. Threats to validity

In this section we describe the external, internal, and construct threats to the validity of our experiments, and the approaches we used to limit the effects that these threats might have.

5.1. External validity

The first threat to validity in this study is the limited number of applications analyzed. While we feel that the three applications are a good representation across types (open source and industrial) as well as differing complexities (small to millions of lines of code), additional data sets would strengthen the results.

Since the primary goal of applying regression testing techniques is to gain economic benefits, the performance characteristics of applying these techniques must also be considered. In the case of some fingerprint computations, the cost of computing the fingerprint scores may be

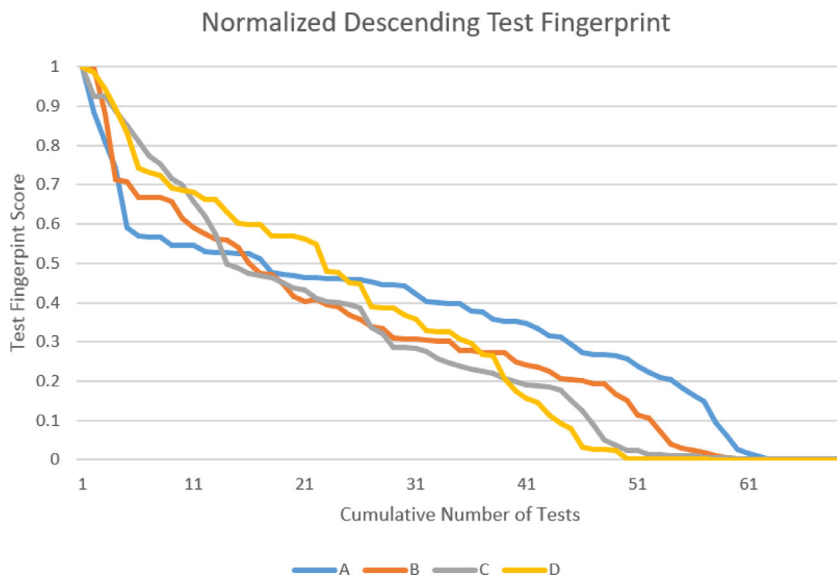


Fig. 29. Test fingerprint scores for CarrotCakeCMS.

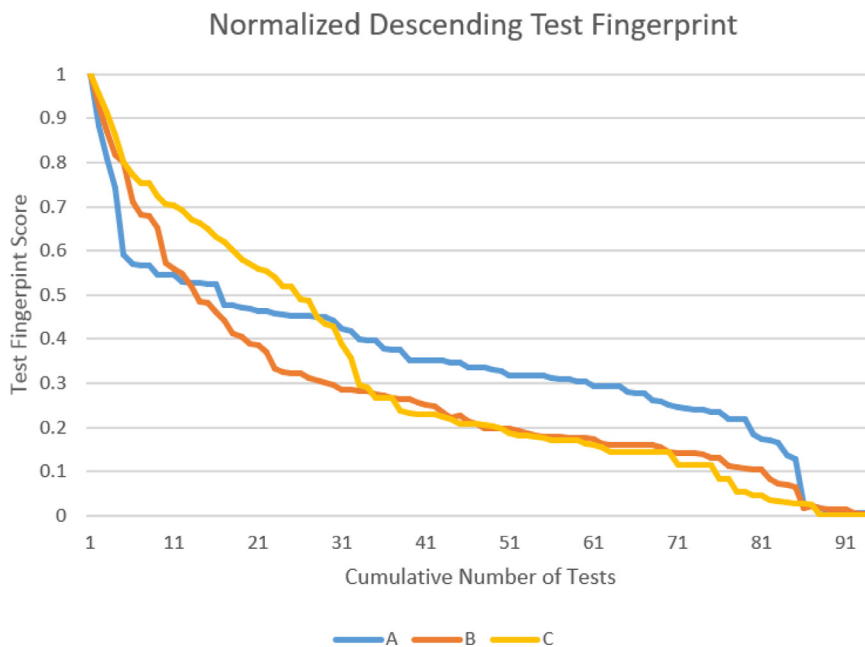


Fig. 30. Test fingerprint scores for DASCP.

significant compared to the cost of executing additional tests, especially in cases where some tests execute in under a second. This can be somewhat mitigated by computing fingerprint scores once and then reusing those scores in subsequent prioritizations. Reusing fingerprints is only valid if the fingerprints do not change over time.

5.2. Internal validity

The accuracy of the collected users' session data from CarrotCakeCMS is another threat to the validity. As we mentioned in Section 4.4.4 we asked the nonprofessional students to perform some basic tasks with this project. Although, we had trained the students to how to work with the system because they were not expert the collected user sessions contains noisy interactions such as duplicate requests, inconsistent interaction patterns, etc. However, we can reduce this threat by monitoring users actions directly or conducting a noise filtering function that can automatically remove the noises from the data.

The time during collection of the datasets was also compressed to a limited range of time. As usage characteristics may change over time, this limited range may have an impact on the accuracy of the predictions. If usage does change over time, a windowing approach to analyzing the dataset may be beneficial as was shown by Anderson et al. [3].

5.3. Construct validity

An obvious criticism of the concept of “discoverable faults” is that all faults are important, and a fault which has not yet been discovered may be run into in the future. True, usage changes over time, and as it changes, the areas of investment in the product similarly change. No reasonably complex software will ever be completely bug free. While we will always strive for that ideal, development effort is pragmatically a finite resource. As such we seek to apply it most efficiently in areas that are most likely to benefit the most customers the soonest. In the future, we would like to expand this research to consider the longer term impact of discoverable faults.

Another threat to validity is the difficulty in determining a proper ground truth. We considered “discoverable faults” to be those that are exercised by users in a subsequent usage period. However, exercising code containing a fault does not always make that fault discoverable by the user. While we feel that this is a better ground truth than the more traditional measure of “all faults”, it still is not perfect.

6. Discussion

In this section, we discuss the implications of these results as well as some of the considerations that must be made in attempting to apply the techniques.

6.1. Improvement in methodology

We believe that the incorporation of usage data into regression testing techniques is a fundamental improvement in current methodology. We are finding that pervasive telemetry is fundamentally changing how we do software engineering and quality assurance at Microsoft, and that trend appears to be replicating itself at most other major software vendors. What has been lacking in both industry and research is tying this new signal of usage telemetry with traditional regression testing techniques such as test case prioritization. In this research, we have shown that the application of usage telemetry to regression testing techniques is possible, is advantageous, and we have demonstrated concrete examples of how it can be applied.

6.2. Churn-based prioritization

One of the most widely used prioritization techniques both in industry and in research is churn-based prioritization [21,39,42,47]. In this technique, tests that execute code that has been changed have higher priority than other tests. This technique is based on the theory that only tests that are executing changed code have the possibility of finding new bugs.

Churn-based prioritization becomes more difficult in cases either where a large number of changes have been made, or where changes have been made to foundational functionality that is executed by a large number of tests. In these cases, churn-based prioritization will conduct a large number of tests and may be less effective than other techniques. These situations arise commonly during large product releases or when important common logic is updated. In both of these situations, we feel that fingerprint-based prioritization offers an effective alternative or a supplement to churn-based prioritization.

6.3. Specialization of workloads

Because fingerprinting is based on the ability to segregate usage patterns, the application of this technique is most successful in situations where usage varies widely [7,19]. For instance, of the datasets described in the first study, one had been generated by a customer who used the software as a platform to build a new application. This customer did not use any of the forms from the base product, so segregating usage based on form name was highly effective. Similarly, another dataset in this same study was from a customer who used only a single module from the product. In cases in which usage does not differ significantly between installations, fingerprinting techniques will be less effective.

While this may seem like a limitation, we believe that widely varying usage patterns is the norm in many applications rather than the exception. In ERP products, virtually no customer uses all of the modules included in the product. Of the modules they do use, the specific forms and controls as well as their patterns of usage will often vary significantly.

6.4. Statistical and practical significance

Our results indicate that the differences between heuristic and control techniques are statistically significant, but the improvements

Table 11

Experiment objects and associated data.

Application	Control	Heuristic	Effect size
Dynamics AX	Greedy	Fingerprinting	0.32
DASCP	Greedy	Fingerprinting	1.67
CarrotCake CMS	Greedy	Fingerprinting	1.16

achieved by the heuristic are not necessarily practically significant. Therefore, we measured the effect sizes of differences to see whether the differences we observed through the statistical analyses are practically meaningful [38]. Table 11 presents the effect sizes of differences between greedy and fingerprinting techniques. As shown in Table 11, the effect sizes range from 0.3 to 1.67, which are considered to be small (Dynamics AX) and large (DASCP and CarrotCake) effect sizes, so we can say that the differences we observed in our studies are indeed practically significant.

6.5. Other applications of techniques

Traditional regression test prioritization, as demonstrated in RQ1 and RQ1', is well understood. We feel that there are other applications of the fingerprinting technique that may be valuable.

One example, used in the *Microsoft Dynamics AX* product, is building canonical usage models for testing. ERP products have widely varying usage patterns depending on the type of industry. One challenge when planning large-scale manual testing activities or building performance testing strategies is deciding what a canonical installation looks like. For example, a retail solution may have hundreds of legal entities, but a small number of corporate purchase orders. On the other hand, a distribution company may only have a single legal entity but millions of purchase orders. By computing fingerprint scores and applying clustering techniques on the resulting values, we can construct canonical clusters of usage patterns that are representative of actual usage.

We could also use fingerprinting techniques to determine the applicability of hotfixes for customers. For instance, if we find a critical bug, examining usage fingerprints would allow us to identify which customers are most likely to benefit from that hotfix. This is important in industries like ERP, where customers are unwilling to take hotfixes once their system is stable unless absolutely necessary.

Further, companies can make better decisions about maintenance and upgrade plans by having a good understanding of how the software is used. For example, if particular system components are accessed by more users, then the company can make more effort to enhance such components, to add more features to them, or to test them with extra caution so that the service that the company provides can attract more users.

Also, in the case of *CarrotCakeCMS*, we learned that some components such as “template-uploading” and “user-profile” are used more frequently by most of users than other components. This means that the user access rates to those particular components are higher, and thus, for future releases, developers should pay greater attention to such components to eliminate potential defects in them. Another lesson we learned from this study is that we can use extracted patterns to define and create more robust components. By knowing users' interests, developers can spend more time and effort upgrading design components that are used by most users and give lower priority to those components that are only rarely used. For instance, from the results of RQ1', we learned that even if the usage patterns are not very coherent, fingerprinting technique still works better than the greedy technique. In particular, in terms of time savings, the results indicate that using fingerprinting we saved 41% and 62% of test execution time in average compared to the greedy technique for *DASCP* and *CarrotCakeCMS*, respectively.

Further, by determining behavioral patterns, we are able to build a customized framework for future software products. By closely

examining user interaction patterns, we discovered some unexpected dependency relations among components. For instance, in *DASCP*, we observed that there is a dependency relationship among a set of components that we did not initially expect 80% of users, after filling out the “civil-project-basic” form, performed three common tasks namely checking out the “alarm date”, “contract status”, and “delivery specification” components. This process seems like a workflow, although it was not defined at the design time and being observed after deployment.

7. Background and related work

In this section, we discuss background information and related work relevant to regression testing techniques and software engineering activities that use telemetry data.

7.1. Regression testing

Regression testing is the process of testing modified software to ensure its continued quality. Typically, regression testing is performed by reusing test cases developed from testing prior versions of the software system and by creating new test cases that will be used to test new features [20,43]. Regression testing techniques of selection and prioritization have been heavily researched for well over 15 years. In selection, tests that are considered to be more important are selected to be run, while those that are less important are skipped. In prioritization, tests are ordered in such a way that faults can be detected earlier, or such that the run may be discontinued early if necessary without significantly reducing detection rates.

To date, various regression testing techniques have been proposed, and recent surveys [13,23,50] provide overviews of many different techniques. More advanced techniques have recently been evaluated, including linear programming [28,30,35], genetic algorithms [34,48], and more advanced classification techniques [4]. Further, several other researchers have applied regression testing techniques to improve fault localization [12,25,26]. For example, Gonzalez-Sanchez et al. [25] presented a diagnostic test prioritization algorithm that reorders test case based on their ability of fault localization. Many of these techniques begin to introduce more dynamic measures such as code churn [8,17,37,42,47], which is an indication of which changes have been made since a previous release.

7.2. Test case prioritization

Test case prioritization techniques reorder test cases to maximize some objective functions, such as detecting defects as early as possible. Due to the appealing benefits of test case prioritization in practice, many researchers have proposed various techniques. These techniques help engineers discover faults early in testing, which allows them to begin debugging earlier. In this case, entire test suites may still be executed, which avoids the potential drawbacks associated with omitting test cases. Recent surveys [13,50] provide a comprehensive understanding of overall trends of the techniques and suggest areas for improvement.

Various test case prioritization technique can be applied depending on the different type of testing goals and available information. Majority of the prioritization techniques have utilized source code and particularly code coverage information to implement the techniques [20,33,44]. Although this approach is naïve, many empirical studies have shown that this approach can be effective [15,16,22,41]. Recent prioritization techniques have used other types of code information, such as slices [31], change history [47], code modification information, and fault proneness of code [36].

7.3. Application of telemetry data in regression testing

Bioinformatics is a growing area of research that uses techniques similar to regression testing, taking large amounts of input data to prioritize

treatments or predict diseases. One area of recent interest is “personalized medicine” [27], in which detailed genomic information from a single person is used to custom tailor treatment for that individual. The analog to bioinformatics genomic data in software is telemetry data.

Telemetry has been used in regression testing, most commonly in the area of graphical user interface (GUI) test generation. This includes research by Brooks and Memon [10] and Amalfitano et al. [2]. Brooks and Memon used telemetry from user interface sessions to generate “usage profiles”, which are the sequences of events from user sessions. These were then used to generate test cases similar to what users had done in practice. Amalfitano et al. similarly collected usage data from rich Internet applications to build finite state models, which were then used in test case generation. Elbaum et al. [19] used telemetry from installations for regression prioritization, but focused on prioritization at a product level. In this research, 1200 user sessions were collected from a 155 KLOC deployed system and then used in regression testing techniques. Their work was extended by others looking at topics like duplicate fault detection [49], performance issue detection [40], reliability testing of rule-based systems [6], and failure reproduction [32].

Barik et al. [7] provide a survey by interviewing Microsoft developers, project managers, and engineers to evaluate how different teams use telemetry data and for what tasks, and moreover to investigate what challenges they face when working with this type of data. Sampath et al. [45] applied several techniques for test case prioritization to investigate which method is fastest in terms of finding faults earlier in web applications. To do this, they examined users session data as a factor of prioritization and showed that frequency metrics and systematic coverage of parameters-value interactions provide better results than other techniques. Fu et al. [24] provide an industrial study of where Microsoft developers use logging. Their purpose is to help developers make briefed decisions of where and how to use logging. Busany and Maoz [11] provide a statistical analysis of behavioral logging from trace sampling to address the scalability issues of large logs.

Our research seeks to apply traditional regression testing techniques based on telemetry data. The difference between our work and existing work is that we wish to apply those techniques using the newly available telemetry from the Software-as-a-Service environment in order to personalize the prioritization to a particular installation. To our knowledge, this method of prioritization is novel and has not yet been explored in research.

8. Conclusions

In this research, we have proposed a novel concept of fingerprinting software usage in order to improve regression testing techniques. To investigate our approach, we performed empirical studies using one large industrial application and two web applications. The empirical results indicate that fingerprint algorithms based on usage telemetry can be identified, that those fingerprints are effective in identifying usage patterns, and that the fingerprints can be applied to improve regression testing techniques.

By applying these techniques, we believe that regression testing techniques can be improved beyond the current state-of-the-art, yielding additional cost and quality benefits. If software testing can be customized to the usage in a particular installation, downtime costs can be reduced while quality is retained. Further, by focusing on faults that users will actually encounter, those faults can be proactively addressed, leading to better user experience and improved product quality.

For future work, we wish to continue expanding our research in combining usage information with regression testing techniques. Further, there are many dimensions to regression testing techniques, some of which may prioritize tests differently than usage-based prioritization would. For instance, fault severity is one measure other research has focused on when prioritizing software quality activities. We would like to combine fault severity with discoverability to provide even better measures. In cases where there are differences in ordering, we hope

to develop a better qualitative understanding of how to select the best techniques and data sources.

Conflict of interest

None.

Acknowledgment

This work was supported, in part, by NSF CAREER Award CCF-1564238 to University of North Texas.

References

- [1] (<https://carrotcakecms.codeplex.com>). (Accessed: 26 April 2016) 2016.
- [2] D. Amalfitano, A.R. Fasolino, P. Tramontana, Rich internet application testing using execution trace data, in: Proceedings of the Software Testing, Verification, and Validation Workshops, 2010, pp. 274–283.
- [3] J. Anderson, S. Salem, H. Do, Improving the effectiveness of test suite through mining historical data, in: Proceedings of the Working Conference on Mining Software Repositories, 2014, pp. 142–151.
- [4] J. Anderson, S. Salem, H. Do, Striving for failure: an industrial case study about test failure prediction, in: Proceedings of the 37th IEEE International Conference on Software Engineering, 2015.
- [5] J. Anderson, S. Salem, H. Do, Customized regression testing using telemetry usage patterns, in: Proceedings of the 32nd IEEE International Conference on Software Maintenance and Evolution, 2016.
- [6] A. Avritzer, J.P. Ros, E.J. Weyuker, Reliability testing of rule-based systems, *IEEE Softw.* 13 (5) (1996) 76.
- [7] T. Barik, R. DeLine, S. Drucker, D. Fisher, The bones of the system: a case study of logging and telemetry at microsoft, in: Proceedings of the International Conference of Software Engineering (ICSE), 2016.
- [8] Á. Beszédes, T. Gergely, L. Schrettnner, J. Jász, L. Langó, T. Gyimóthy, Code coverage-based regression test selection and prioritization in webkit, in: Proceedings of the 2012 28th IEEE International Conference on Software Maintenance (ICSM), IEEE, 2012, pp. 46–55.
- [9] R.V. Binder, *Testing Object-Oriented Systems*, Addison Wesley, Upper Saddle River, NJ, 1999.
- [10] P.A. Brooks, A.M. Memon, Automated gui testing guided by usage profiles, in: Proceedings of the Twenty-second IEEE/ACM international Conference on Automated Software Engineering, 2007, pp. 333–342.
- [11] N. Busany, S. Maoz, Behavioral log analysis with statistical guarantees, in: Proceedings of the International Symposium on Foundations of Software Engineering (ESEC/FSE), 2015.
- [12] J. Campos, R. Abreu, G. Fraser, M. d Amorim, Entropy-based test generation for improved fault localization, in: Proceedings of the International Conference on Automated Software Engineering (ASE), 2013.
- [13] C. Catal, D. Mishra, Test case prioritization: a systematic mapping study, *Softw. Qual. J.* 21 (2013) 445–478.
- [14] D. Cotroneo, M. Grottke, R. Natella, R. Pietrantuono, K.S. Trivedi, Fault triggers in open-source software: an experience report, in: Proceedings of the 2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE), IEEE, 2013, pp. 178–187.
- [15] H. Do, S. Mirarab, L. Tahvildari, G. Rothermel, An empirical study of the effect of time constraints on the cost-benefits of regression testing., in: Proceedings of the ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE), IEEE-ACM, 2008.
- [16] H. Do, S. Mirarab, L. Tahvildari, G. Rothermel, The effects of time constraints on test case prioritization: a series of controlled experiments, *IEEE Trans. Softw. Eng.* 36 (5) (2010).
- [17] H. Do, S. Mirarab, L. Tahvildari, G. Rothermel, The effects of time constraints on test case prioritization: a series of controlled experiments, *IEEE Trans. Softw. Eng.* 26 (5) (2010).
- [18] H. Do, G. Rothermel, A. Kinneer, Prioritizing JUnit test cases: an empirical assessment and cost-benefits analysis, *Empir. Softw. Eng. Int. J.* 11 (1) (2006) 33–70.
- [19] S. Elbaum, M. Diep, Profiling deployed software: assessing strategies and testing opportunities, *IEEE Trans. Softw. Eng.* 31 (4) (2005) 312–327.
- [20] S. Elbaum, A.G. Malishevsky, G. Rothermel, Test case prioritization: a family of empirical studies, *IEEE Trans. Softw. Eng.* 28 (2) (2002) 159–182.
- [21] S. Elbaum, J. Munson, Code churn: a measure for estimating the impact of code change, in: Proceedings of the International Conference on Software Maintenance, 1998, pp. 24–31.
- [22] S. Elbaum, A.G. Malishevsky, G. Rothermel, Test case prioritization: a family of empirical studies, *IEEE Trans. Softw. Eng.* 28 (2) (2002) 159–182.
- [23] E. Engstrom, P. Runeson, M. Skoglund, A systematic review on regression test selection techniques, *Inf. Softw. Technol.* 52 (1) (2010) 14–30.
- [24] Q. Fu, J. Zhu, W. Hu, J. Lou, R. Ding, Q. Lin, D. Zhang, T. Xie, Where do developers log? An empirical study on logging practices in industry, in: Proceedings of the International Conference of Software Engineering (ICSE), 2014.
- [25] A. Gonzalez-Sanchez, R. Abreu, H.-G. Gross, A.J. van Gemund, Prioritizing tests for fault localization through ambiguity group reduction, in: Proceedings of the International Conference on Automated Software Engineering (ASE), 2011.
- [26] A. Gonzalez-Sanchez, R.A. breu, Spectrum – based sequential diagnosis, in: Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence, 2011.
- [27] M.A. Hamburg, F.S. Collins, The path to personalized medicine, *N. Engl. J. Med.* 363 (4) (2010) 301–304.
- [28] D. Hao, L. Zhang, X. Wu, H. Mei, G. Rothermel, On-demand test suite reduction, in: Proceedings of the 34th International Conference on Software Engineering, 2012, pp. 738–748.
- [29] L. Hatton, Reexamining the fault density component size connection, *IEEE Softw.* 14 (2) (1997) 89–97.
- [30] S. Hou, L. Zhang, T. Xie, J. Sun, Quota-constrained test case prioritization for regression testing of service-centric systems, in: Proceedings of the International Conference on Software Maintenance, 2008, pp. 257–266.
- [31] D. Jeffrey, N. Gupta, Test case prioritization using relevant slices, in: Proceedings of the International Computer Software and Applications Conference, 2006, pp. 411–420.
- [32] W. Jin, A. Orso, Bugredux: reproducing field failures for in-house debugging, in: Proceedings of the 34th International Conference on Software Engineering, IEEE Press, 2012, pp. 474–484.
- [33] J. Kim, A. Porter, A history-based test prioritization technique for regression testing in resource constrained environments, in: Proceedings of the International Conference on Software Engineering, 2002.
- [34] Z. Li, M. Harman, R. Hierons, Search algorithms for regression test case prioritization, *IEEE Trans. Softw. Eng.* 33 (4) (2007) 225–237.
- [35] S. Mirarab, S. Akhlaghi, L. Tahvildari, Size-constrained regression test case selection using multicriteria optimization, *IEEE Trans. Softw. Eng.* 38 (4) (2012) 936–956.
- [36] S. Mirarab, L. Tahvildari, A prioritization approach for software test cases on Bayesian Networks, in: Proceedings of the Fundamental Approaches to Software Engineering, 2007, pp. 276–290.
- [37] A. Mockus, N. Nagappan, T.T. Dinh-Trong, Test coverage and post-verification defects: a multiple case study, in: Proceedings of the 2009 3rd International Symposium on Empirical Software Engineering and Measurement, IEEE, 2009, pp. 291–301.
- [38] C. Murphy, B. Myers, *Statistical Power Analysis: A Simple and General Model for Traditional and Modern Hypothesis Tests*, Lawrence Erlbaum Associates, 1998.
- [39] N. Nagappan, T. Ball, Using software dependencies and churn metrics to predict field failures: an empirical case study, in: Proceedings of the ESEM, IEEE, 2007, pp. 364–373.
- [40] T. Parsons, Automatic detection of performance design and deployment antipatterns in component based enterprise systems, Citeseer, 2007 Ph.D. thesis.
- [41] X. Qu, M. Cohen, G. Rothermel, Configuration-aware regression testing: an empirical study of sampling and prioritization, in: Proceedings of the International Symposium on Software Testing and Analysis (ISSTA), IEEE-ACM, 2008, pp. 75–85.
- [42] G. Rothermel, S. Elbaum, A.G. Malishevsky, P. Kallakuri, X. Qiu, On test suite composition and cost-effective regression testing, *ACM Trans. Softw. Eng. Methodol.* 13 (3) (2004) 227–331.
- [43] G. Rothermel, R. Untch, C. Chu, M.J. Harrold, Test case prioritization: an empirical study, in: Proceedings of the International Conference on Software Maintenance, 1999, pp. 179–188.
- [44] G. Rothermel, R. Untch, C. Chu, M.J. Harrold, Prioritizing test cases for regression testing, *IEEE Trans. Softw. Eng.* 27 (10) (2001) 929–948.
- [45] S. Sampath, C.B. Renee, G. Viswanath, V. Kandimalla, A.G. Koru, Prioritizing user-session-based test cases for web applications testing, in: Proceedings of the International Conference of Software Testing (ICST), 2008.
- [46] S. Sampath, R. Bryce, G. Viswanath, V. Kandimalla, A. Koru, Prioritizing user-session-based test cases for web applications testing, in: Proceedings of the International Conference on Software Testing, Verification, and Validation, 2008, pp. 141–150.
- [47] M. Sherriff, M. Lake, L. Williams, Prioritization of regression tests using singular value decomposition with empirical change records, in: Proceedings of the International Symposium on Software Reliability Engineering, 2007, pp. 81–90.
- [48] K. Walcott, G. Kapfhammer, R. Roos, M.L. Soffa, Time-aware test suite prioritization, in: Proceedings of the International Symposium on Software Testing and Analysis, 2006.
- [49] X. Wang, L. Zhang, T. Xie, J. Anvik, J. Sun, An approach to detecting duplicate bug reports using natural language and execution information, in: Proceedings of the International Conference on Software Engineering, 2008, pp. 461–470.
- [50] S. Yoo, M. Harman, Regression testing minimisation, selection and prioritisation : a survey, *Softw. Test. Verif. Reliab.* 22 (2) (2012) 67–120.